# COMP3711: Design and Analysis of Algorithms

WU Jinjun

October 12, 2026

## Contents

**1 Prerequisite: Asymptotic Notations** **4**

**2 Divide and Conquer** **5**

2.1 Binary Search . . . . . 5

2.1.1 Variant: Find the Boundary (lower_bound) . . . . . 6

2.1.2 Exercise 1: Rotated Sorted Array . . . . . 7

2.1.3 Exercise 2: Finding the Last 0 . . . . . 8

2.2 Mergesort & Inversion Count . . . . . 8

2.2.1 Warm-up: Tower of Hanoi . . . . . 8

2.2.2 Merge: Combining Two Sorted Halves . . . . . 9

2.2.3 Mergesort Algorithm . . . . . 10

2.2.4 Counting Inversions . . . . . 11

2.3 Maximum Subarray . . . . . 12

2.3.1 Application: Maximizing Stock Profit . . . . . 14

2.4 Multiplication and Master Theorem . . . . . 14

2.4.1 Integer Multiplication (Karatsuba) . . . . . 14

2.4.2 First D&C Attempt: Four Half-Size Products . . . . . 14

2.4.3 Master Theorem for Recurrences . . . . . 16

2.5 Matrix Multiplication and Strassen's Algorithm . . . . . 17

**3 Randomized Algorithms** **18**

3.1 Why Randomization? . . . . . 18

3.2 Probability Review . . . . . 18

3.3 Linearity of Expectation . . . . . 18

3.4 Example: Hiring Problem . . . . . 19

3.5 More Indicator-Variable Examples . . . . . 19

3.5.1 Guessing Shuffled Cards . . . . . 19

3.5.2 Hash Table with Chaining . . . . . 19

3.5.3 Birthday Paradox . . . 20
3.6 Generating a Uniform Random Permutation . . . 20
3.7 Geometric Distribution and Coupon Collector . . . 20
3.8 Case Study: The Majority Element . . . 20

**4 Quicksort** **22**
4.1 Deterministic Quicksort (Lomuto Partition) . . . 22
4.2 Quicksort Analysis . . . 23
4.3 Randomized Selection (Quickselect) . . . 24
4.4 Hoare's Partition (Alternative) . . . 25

**5 Heapsort** **26**
5.1 Priority Queue Motivation . . . 26
5.2 Heap Data Structure . . . 26
5.3 Max-Heapify (Bubble-Down) . . . 26
5.4 Build Max-Heap . . . 27
5.5 Heapsort Algorithm . . . 28
5.6 Priority Queue Operations (Applications of Heaps) . . . 28
5.7 Applications . . . 29
5.7.1 Merging $k$ Sorted Arrays . . . 29
5.7.2 Delete and Update-Priority . . . 29
5.7.3 The Skyline Problem . . . 29

**6 Linear-Time Sorting** **30**
6.1 The Decision-Tree Model and the Sorting Lower Bound . . . 30
6.2 Counting Sort . . . 31
6.3 Radix Sort . . . 31
6.4 Bucket Sort . . . 32

**7 Greedy Algorithms** **34**
7.1 Warm-up: Change-Making . . . 34
7.2 Warm-up: Max Sum of Non-Adjacent Elements . . . 34
7.3 Interval Scheduling . . . 35
7.3.1 Three Tempting Greedy Rules (All Fail) . . . 35
7.3.2 The Correct Rule: Earliest Finish Time . . . 35
7.3.3 Correctness Proof (Exchange Argument) . . . 36
7.4 Exercise: Interval Covering . . . 36
7.5 Fractional Knapsack . . . 37
7.6 Interval Partitioning . . . 38

**8 Huffman Coding** **40**

8.1 Motivation: Variable-Length Encoding . . . . . 40

8.2 Prefix-Free Codes . . . . . 40

8.3 The Encoding Problem . . . . . 40

8.4 Tree Correspondence and Weighted External Path Length . . . . . 41

8.5 Huffman's Greedy Algorithm . . . . . 41

8.6 Worked Example . . . . . 41

8.7 Correctness Proof . . . . . 42

8.8 Application: Optimal Two-Way Merge . . . . . 43

# 1 Prerequisite: Asymptotic Notations

$O, \Omega, \Theta$ are used to describe the asymptotic behavior of functions.

## Definitions of Asymptotic Notations

(a) **Big-O (Upper Bound):** $f(n) = O(g(n))$ means that there exist positive constants $c$ and $n_0$ such that $f(n) \leq cg(n)$ for all $n \geq n_0$.

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} < \infty$$

(b) **Big-Omega (Lower Bound):** $f(n) = \Omega(g(n))$ means that there exist positive constants $c$ and $n_0$ such that $f(n) \geq cg(n)$ for all $n \geq n_0$.

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} > 0$$

(c) **Big-Theta (Tight Bound):** $f(n) = \Theta(g(n))$ means that there exist positive constants $c_1, c_2$ and $n_0$ such that $c_1 g(n) \leq f(n) \leq c_2 g(n)$ for all $n \geq n_0$.

$$0 < \lim_{n\to\infty} \frac{f(n)}{g(n)} < \infty$$

(d) **Not Comparable:** $f(n)$ and $g(n)$ are not comparable if $\lim_{n\to\infty} \frac{f(n)}{g(n)}$ does not exist.

## Asymptotic Growth Hierarchy (Memorize This Order)

constant $<$ logarithmic $<$ polynomial $<$ exponential

$$O(1) \subset O(\log\log n) \subset O(\log n) \subset O(\sqrt{n}) \subset O(n) \subset O(n\log n)$$
$$\subset O(n^2) \subset O(n^k) \subset O(2^n) \subset O(n!) \subset O(n^n)$$

## Common Mistakes with Asymptotic Notation

- $O(g(n))$ describes an **upper bound**, not necessarily tight. $n = O(n^2)$ is technically true but not useful.
- $\Omega(g(n))$ describes a **lower bound**. Best-case $\neq$ lower bound!
- $\Theta(g(n))$ requires **both** upper and lower bounds to match.
- Always drop constant factors and lower-order terms: $3n^2 + 5n + 7 = \Theta(n^2)$, not $\Theta(3n^2)$.

# 2 Divide and Conquer

Solve a problem of size $n$ by breaking it into one or more smaller problems of size less than $n$. Solve the smaller problems recursively and combine their solutions to solve the large problem.

**Three Steps of D&C**

1. **Divide:** Split the problem into subproblems.
2. **Conquer:** Solve subproblems recursively (or directly if small enough).
3. **Combine:** Merge subproblem solutions into the original solution.

## 2.1 Binary Search

Given a sorted array $A$ and a target value $x$, find the index of $x$ (or $-1$ if not found).

**Algorithm 1** Binary Search

```
function BinarySearch(A, x)
    low ← 0
    high ← length(A) − 1
    while low ≤ high do
        mid ← ⌊(low + high)/2⌋
        if A[mid] = x then
            return mid
        else if A[mid] < x then
            low ← mid + 1
        else
            high ← mid − 1
        end if
    end while
    return −1
end function
```

**Binary Search Analysis**

Let $T(n)$ be the number of comparisons on an array of size $n$.

- Recurrence: $T(n) \leq T\left(\frac{n}{2}\right) + 2$, with $T(1) = T(0) = 1$.
- Expand: $T(n) \leq T\left(\frac{n}{2}\right) + 2 \leq T\left(\frac{n}{4}\right) + 4 \leq \cdots \leq T(1) + 2\log_2 n = 1 + 2\log_2 n$.
- Therefore: $T(n) = O(\log n)$.

**Algorithm 2** Binary Search (Recursive Version)

```
function BinarySearch(A, l, r, x)
    if l > r then
        return nil                                   ▷ x not present
    end if
    m ← ⌊(l + r)/2⌋
    if A[m] = x then
        return m
    else if x < A[m] then
        return BinarySearch(A, l, m − 1, x)
    else
        return BinarySearch(A, m + 1, r, x)
    end if
end function
```

**Worked example:** search for $x = 42$ in

$$A = [4, 7, 10, 15, 19, 20, 42, 54, 87, 90].$$

| Call | Range | $m$ | $A[m]$ vs. 42 |
|---|---|---|---|
| 1 | $[1, 10]$ | 5 | $19 < 42 \Rightarrow$ go right |
| 2 | $[6, 10]$ | 8 | $54 > 42 \Rightarrow$ go left |
| 3 | $[6, 7]$ | 6 | $20 < 42 \Rightarrow$ go right |
| 4 | $[7, 7]$ | 7 | $A[7] = 42$ **found!** |

**Recursion-Tree View and the $\Omega(\log n)$ Lower Bound**

The recursion tree is a single path: exactly one subproblem per level, costing 2 comparisons per internal level and 1 at the leaf, giving $2\log_2 n + 1$ comparisons — equivalent to expansion but more visual.

- Upper bound: each level halves the range, so at most $\lfloor \log_2 n \rfloor + 1$ levels $\Rightarrow T(n) = O(\log n)$.
- Lower bound: an element sitting at a deepest leaf (or a failed search) forces traversing the full path of length $\log_2 n$, so $T(n) = \Omega(\log n)$.

Hence $T(n) = \Theta(\log n)$. More generally, the decision-tree model proves no search algorithm on a sorted array can beat $\Omega(\log n)$ comparisons (there are $n + 1$ possible answers: $n$ positions plus "not found").

### 2.1.1 Variant: Find the Boundary (lower_bound)

Consider the variant with **no equality test**:

**Algorithm 3** Binary Search without Equality Test

```
function FindBoundary(A, l, r, x)
    if l = r then
        return l
    end if
    m ← ⌊(l + r)/2⌋
    if A[m] < x then
        return FindBoundary(A, m + 1, r, x)
    else
        return FindBoundary(A, l, m, x)
    end if
end function
```

**What Does This Variant Return?**

- It returns the index of the **first** element $\geq x$ — the insertion position even if $x$ is absent (C++ `std::lower_bound`, Python `bisect.bisect_left`).
- Changing $A[m] < x$ to $A[m] \leq x$ returns the first element **strictly greater** than $x$ (`upper_bound`, `bisect_right`); the two answers differ exactly when $x$ occurs in $A$.
- Running time is still $O(\log n)$: every call halves a non-empty range.

**Tail recursion into a loop.** Because the recursive call is the very last action (tail recursion), it rewrites directly as a `while` loop:

```
l = 1; r = n;
while (l < r) {
    m = (l + r) / 2;
    if (A[m] < x) l = m + 1;
    else          r = m;
}
return l;
```

The iterative form avoids function-call overhead, uses $O(1)$ instead of $O(\log n)$ space, and cannot overflow the call stack — this is why library routines are written this way.

**When to Use Binary Search**

Binary search works whenever you can discard half the search space with one comparison. The array need not be literally sorted—it just needs a "monotonic" property that lets you decide which half contains the answer.

### 2.1.2 Exercise 1: Rotated Sorted Array

Let $A[1..n]$ be a sorted array of $n$ distinct numbers that has been rotated $n-k$ steps for some unknown integer $k \in [1, n-1]$. That is, $A[1..k]$ is sorted increasing, $A[k+1..n]$ is sorted increasing, and $A[n] < A[1]$.

**Example:** $n = 16$, $k = 10$:

$$A = [9, 13, 16, 18, 19, 23, 28, 31, 37, 42, 0, 1, 2, 5, 7, 8]$$

**(1) Find the rotation index $k$ in $O(\log n)$.** ($A[k]$ is the maximum element.)

**Algorithm 4** Find Rotated Index

**function** FindRotatedIndex($A, l, r$)
  $mid \leftarrow \left\lfloor \frac{l+r}{2} \right\rfloor$
  **if** $A[mid] > A[mid+1]$ **then**
    **return** $mid$
  **else if** $A[mid] \geq A[l]$ **then**
    **return** FindRotatedIndex($A, mid+1, r$)
  **else**
    **return** FindRotatedIndex($A, l, mid-1$)
  **end if**
**end function**

**(2) Search for $x$ in $O(\log n)$.**

**Algorithm 5** Search in Rotated Sorted Array

```
function SearchRotated(A, x, n)
    k ← FindRotatedIndex(A, 1, n)
    if x ≥ A[1] then
        return BinarySearch(A[1..k], x)
    else
        return BinarySearch(A[k + 1..n], x)
    end if
end function
```

### 2.1.3 Exercise 2: Finding the Last 0

Array $A[1..n]$ contains 0s followed by 1s.

**(1) Standard $O(\log n)$ solution.**

**Algorithm 6** Find Last 0

```
function FindLast0(A, l, r)
    mid ← ⌊(l + r)/2⌋
    if A[mid] = 0 and A[mid + 1] = 1 then
        return mid
    else if A[mid] = 0 then
        return FindLast0(A, mid + 1, r)
    else
        return FindLast0(A, l, mid − 1)
    end if
end function
```

**(2) $O(\log k)$ solution when $k \ll n$.** Exponential search (gallop): double the index until we overshoot, then binary search.

**Algorithm 7** Find Last 0 with $O(\log k)$

```
i ← 1
while i < n ∧ A[i] = 0 do
    i ← min{2i, n}
end while
if A[i] = 0 then
    return i                          ▷ all zeros: last 0 is n
else if i = 1 then
    return −1                         ▷ no zero
else
    return FindLast0(A, ⌊i/2⌋, i)
end if
```

## 2.2 Mergesort & Inversion Count

### 2.2.1 Warm-up: Tower of Hanoi

Three pegs, $n$ disks of decreasing size. Move all disks from source to target peg.

**Hanoi Recurrence**

Let $T(n)$ = minimum moves for $n$ disks.

- Basis: $T(1) = 1$.
- Recursive step: move $n - 1$ disks source→aux, move largest source→target, move

$n-1$ aux→target.

$$T(n) = 2T(n-1) + 1$$

- Solution: $T(n) = 2^n - 1$. (Expand: $T(n) = 2^k T(n-k) + (2^k - 1)$, set $k = n-1$.)

**Algorithm 8** Hanoi Tower

```
function HANOI(n, A, B, C)
    if n = 1 then
        Move disk from A to C
    else
        HANOI(n − 1, A, C, B)
        Move disk from A to C
        HANOI(n − 1, B, A, C)
    end if
end function
```

### 2.2.2 Merge: Combining Two Sorted Halves

Given two sorted subarrays $A[p..q]$ and $A[q+1..r]$, merge them into $A[p..r]$ sorted.

**Algorithm 9** Merge

```
function MERGE(A, p, q, r)
    n1 ← q − p + 1;    n2 ← r − q
    Create arrays L[1..n1 + 1], R[1..n2 + 1]
    for i = 1 to n1 do
        L[i] ← A[p + i − 1]
    end for
    for j = 1 to n2 do
        R[j] ← A[q + j]
    end for
    L[n1 + 1] ← ∞;    R[n2 + 1] ← ∞                ▷ sentinels
    i ← 1;    j ← 1
    for k = p to r do
        if L[i] ≤ R[j] then
            A[k] ← L[i];    i ← i + 1
        else
            A[k] ← R[j];    j ← j + 1
        end if
    end for
end function
```

**Merge Time Complexity**

Merge runs in $\Theta(n)$ time on $n = r-p+1$ elements: two copy loops ($n_1+n_2 = n$) plus one comparison loop ($n$ iterations). Sentinels avoid boundary checks — clean but not cache-friendly in practice.

**Worked example:** merge the sorted halves $L = [2, 4, 5, 7, \infty]$ and $R = [1, 2, 3, 6, \infty]$:

| $k$ | compare | winner | $A[p \dots r]$ so far |
|---|---|---|---|
| 1 | $L[1] = 2$ vs $R[1] = 1$ | $R[1]$ | [1] |
| 2 | $L[1] = 2$ vs $R[2] = 2$ | $L[1]$ ($\leq$ keeps it stable) | [1, 2] |
| 3 | $L[2] = 4$ vs $R[2] = 2$ | $R[2]$ | [1, 2, 2] |
| 4 | $L[2] = 4$ vs $R[3] = 3$ | $R[3]$ | [1, 2, 2, 3] |
| 5 | $L[2] = 4$ vs $R[4] = 6$ | $L[2]$ | [1, 2, 2, 3, 4] |
| 6 | $L[3] = 5$ vs $R[4] = 6$ | $L[3]$ | [1, 2, 2, 3, 4, 5] |
| 7 | $L[4] = 7$ vs $R[4] = 6$ | $R[4]$ | [1, 2, 2, 3, 4, 5, 6] |
| 8 | $L[4] = 7$ vs $\infty$ | $L[4]$ | [1, 2, 2, 3, 4, 5, 6, 7] |

Each step advances the head pointer ($i$ or $j$) of the list whose element was copied; exactly one element is placed per iteration.

### 2.2.3 Mergesort Algorithm

**Algorithm 10** Mergesort

```
function MERGESORT(A, p, r)
    if p < r then
        q ← ⌊(p + r)/2⌋                ▷ Divide
        MERGESORT(A, p, q)             ▷ Conquer left
        MERGESORT(A, q + 1, r)         ▷ Conquer right
        MERGE(A, p, q, r)              ▷ Combine
    end if
end function
```

**Mergesort Analysis**

Recurrence for $n$ elements (power of 2 for simplicity):

$$T(n) = \begin{cases} 1 & \text{if } n = 1, \\ 2T\left(\frac{n}{2}\right) + \Theta(n) & \text{if } n > 1. \end{cases}$$

Solution: $T(n) = \Theta(n \log n)$. (Master Theorem: $a = 2, b = 2, f(n) = \Theta(n)$, case 2.)

**Worked example:** $A = [5, 2, 4, 7, 1, 3, 2, 6]$.

| Stage | Array state |
|---|---|
| input | $[5, 2, 4, 7 \mid 1, 3, 2, 6]$ |
| after sorting left half recursively | $[2, 4, 5, 7 \mid 1, 3, 2, 6]$ |
| after sorting right half recursively | $[2, 4, 5, 7 \mid 1, 2, 3, 6]$ |
| after $\Theta(n)$ merge | $[1, 2, 2, 3, 4, 5, 6, 7]$ |

**Solving the Recurrence by Expansion / Recursion Tree**

Expanding directly:

$$\begin{aligned} T(n) &= 2T(n/2) + n = 2^2T(n/4) + 2n = 2^3T(n/8) + 3n \\ &= 2^iT(n/2^i) + in = nT(1) + n\log_2 n = n\log n + n. \end{aligned}$$

On the recursion tree, level $i$ has $2^i$ nodes, each paying $n/2^i$, so the cost of every level is $2^i \cdot (n/2^i) = n$; with $\log_2 n + 1$ levels the total is $n(\log_2 n + 1)$.

**Why $\Omega(n \log n)$ Too?**

The merge step costs $\Theta(n)$ on every input — there is no lucky case that skips work. Replacing each $O$ by $\Omega$ in the analysis is still valid, so $T(n) = \Omega(n \log n)$ and overall $T(n) = \Theta(n \log n)$: every input is a worst-case input for mergesort.

**Mergesort Properties**

- **Stable:** yes (preserves order of equal keys if $L[i] \leq R[j]$ uses $\leq$, not $<$).
- **In-place:** no (needs auxiliary arrays $L$ and $R$; $\Theta(n)$ extra space).
- **Best / Worst / Average:** all $\Theta(n \log n)$ (unlike Quicksort).

### 2.2.4 Counting Inversions

An **inversion** is a pair $(i, j)$ with $i < j$ and $A[i] > A[j]$. Count the total number of inversions.

**Brute Force is Too Slow**

Checking all $\binom{n}{2}$ pairs takes $\Theta(n^2)$ time. With D&C we can do $\Theta(n \log n)$ by **piggybacking on Mergesort**.

**Algorithm 11** Count Inversions via Mergesort

```
function CountInv(A, p, r)
    if p ≥ r then
        return 0
    end if
    q ← ⌊(p + r)/2⌋
    left ← CountInv(A, p, q)
    right ← CountInv(A, q + 1, r)
    cross ← MergeCount(A, p, q, r)
    return left + right + cross
end function

function MergeCount(A, p, q, r)
    ... same as Merge, but whenever R[j] is copied before L[i] is exhausted:
    add (n_1 − i + 1) to cross (all remaining L[i..n_1] are > R[j])
    return cross
end function
```

**Why Cross-Inversions Work**

When copying $R[j]$ before $L[i]$ is empty, **every remaining element in $L$ forms an inversion with $R[j]$** because $L$ indices $< R$ indices (by array split) and $L[i..n_1] > R[j]$ (since we picked $R[j]$ first). This adds exactly $n_1 - i + 1$ inversions in $O(1)$.

**Applications.** The inversion number measures (i) how "sorted" an array is (0 = sorted, $\binom{n}{2}$ = reverse sorted), and (ii) the similarity between two rankings: map one ranking to the identity permutation and count inversions in the other.

**Worked example:** $A = [4, 8, 10, 2, 1, 5, 12, 11, 3, 7, 6, 9]$.

Split into blue half $[4, 8, 10, 2, 1, 5]$ and green half $[12, 11, 3, 7, 6, 9]$:

- 5 blue-blue inversions and 8 green-green inversions (found recursively).
- 9 blue-green inversions: $(5, 3)$, $(4, 3)$, $(8, 6)$, $(8, 3)$, $(8, 7)$, $(10, 6)$, $(10, 9)$, $(10, 3)$, $(10, 7)$.
- Total: $5 + 8 + 9 = \mathbf{22}$.

**Counting cross inversions during the merge.** With 1-indexed arrays split at $m$, when $R[j]$ is copied while the $L$-pointer is at $i$, the blue elements already passed ($L[1..i-1]$) are $\le R[j]$; every remaining blue element $L[i..m]$ is $> R[j]$, so the contribution of $R[j]$ is

$$I[j] = m - i + 1 \quad \big(\text{or, with the slide's indexing, } m - l - i + 2\big).$$

**Tiny trace:** sorted halves $L = [1, 3, 5, 7]$, $R = [2, 4, 6, 8]$:

| Element copied from $R$ | pointer $i$ | $L[i..]$ greater than it | $I[j]$ |
|---|---|---|---|
| 2 | 1 ($L[1] = 1 \le 2$ passed, then $i = 2$) | 3, 5, 7 | 3 |
| 4 | 3 | 5, 7 | 2 |
| 6 | 4 | 7 | 1 |
| 8 | past end | — | 0 |

Cross inversions $= 3+2+1+0 = 6$; the merge simultaneously produces $[1, 2, 3, 4, 5, 6, 7, 8]$, preserving the sortedness invariant needed by the parent call.

**Recursion-tree accounting.** Each leaf returns 0; each internal node returns its cross count from the merge, and a node reports the sum of its two children plus its own cross count. For the 12-element example the two size-6 root children return 5 (blue half) and 8 (green half), and the root merge contributes 9:

root: $5 + 8 + 9 = 22$
left child `Sort-and-Count`$[4, 8, 10, 2, 1, 5] \to 5$
right child `Sort-and-Count`$[12, 11, 3, 7, 6, 9] \to 8$
(their values are sums of cross counts at lower levels; leaves contribute 0)

Since both the counting and the merging are linear scans, `Merge-and-Count` costs $\Theta(n)$ and the overall recurrence is again $T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n)$.

**Algorithm 12** Merge-and-Count (precondition: $A[l..m]$ and $A[m+1..r]$ sorted)

```
copy A[l..m] into L[1..n1], A[m+1..r] into R[1..n2], append ∞ sentinels
i ← 1;  j ← 1;  c ← 0
for k = l to r do
    if L[i] ≤ R[j] then
        A[k] ← L[i];  i ← i + 1
    else
        A[k] ← R[j];  j ← j + 1
        c ← c + (n1 − i + 1)            ▷ all remaining L elements invert R[j]
    end if
end for
return c
```

## 2.3 Maximum Subarray

Given an array $A[1..n]$, find indices $i, j$ maximizing $\sum_{t=i}^{j} A[t]$ (the empty subarray is not allowed). (Useful for best-buy/best-sell stock problem.)

**Motivating example.** Yearly profits (M$) $[3, 2, 1, -7, 5, 2, -1, 3, -1]$: the best span is years 5–8 with $5 + 2 - 1 + 3 = 9$ M$.

**Two Simpler Solutions First**

1. **Brute force $\Theta(n^3)$:** enumerate all pairs $(i, j)$ and re-sum $A[i..j]$ with an inner loop. The $j$- and $k$-loops average $n/2$ and $n/3$ iterations.
2. **Data reuse $\Theta(n^2)$:** exploit $V(i, j) = V(i, j-1) + A[j]$ — extend the running sum instead of restarting, removing the inner loop.

The D&C solution improves this to $\Theta(n \log n)$, and Kadane's DP reaches $\Theta(n)$.

**D&C Strategy**

For a subarray $A[low..high]$ with midpoint $mid$, the maximum subarray is **exactly one** of:

1. Entirely in $A[low..mid]$ (recurse left).
2. Entirely in $A[mid+1..high]$ (recurse right).
3. **Crossing the midpoint:** one end in left half, one end in right half (compute directly).

Return the maximum of the three.

**Algorithm 13** Maximum Subarray (D&C)

```
function MaxSubarray(A, low, high)
    if low = high then
        return (low, high, A[low])
    end if
    mid ← ⌊(low + high)/2⌋
    (left_lo, left_hi, left_sum) ← MaxSubarray(A, low, mid)
    (right_lo, right_hi, right_sum) ← MaxSubarray(A, mid + 1, high)
    (cross_lo, cross_hi, cross_sum) ← MaxCrossing(A, low, mid, high)
    if left_sum ≥ right_sum and left_sum ≥ cross_sum then
        return (left_lo, left_hi, left_sum)
    else if right_sum ≥ left_sum and right_sum ≥ cross_sum then
        return (right_lo, right_hi, right_sum)
    else
        return (cross_lo, cross_hi, cross_sum)
    end if
end function

function MaxCrossing(A, low, mid, high)
    left_sum ← −∞;    sum ← 0
    for i = mid downto low do
        sum ← sum + A[i]
        if sum > left_sum then
            left_sum ← sum;    max_left ← i
        end if
    end for
    right_sum ← −∞;    sum ← 0
    for j = mid + 1 to high do
        sum ← sum + A[j]
        if sum > right_sum then
            right_sum ← sum;    max_right ← j
        end if
    end for
    return (max_left, max_right, left_sum + right_sum)
end function
```

## Max Subarray Analysis

Recurrence:

$$T(n) = \begin{cases} \Theta(1) & n = 1, \\ 2T(n/2) + \Theta(n) & n > 1. \end{cases}$$

Same as Mergesort: $T(n) = \Theta(n \log n)$.

**Kadane's Algorithm** (iterative DP, not D&C): $O(n)$ time, $O(1)$ space.

$$\text{ending_here}_i = \max\big(A[i],\ \text{ending_here}_{i-1} + A[i]\big),$$
$$\text{best}_i = \max\big(\text{best}_{i-1},\ \text{ending_here}_i\big).$$

**Kadane trace on** $A = [-2, 1, -3, 4, -1, 2, 1, -5, 4]$ (start $V = V_{\max} = -\infty$):

| $i$ | $A[i]$ | $V = \max(V + A[i], A[i])$ (best ending at $i$) | $V_{\max}$ | best window so far |
|---|---|---|---|---|
| 1 | $-2$ | $-2$ | $-2$ | $[-2]$ |
| 2 | 1 | 1 (restart) | 1 | $[1]$ |
| 3 | $-3$ | $-2$ | 1 | $[1]$ |
| 4 | 4 | 4 (restart) | 4 | $[4]$ |
| 5 | $-1$ | 3 | 4 | $[4]$ |
| 6 | 2 | 5 | 5 | $[4, -1, 2]$ |
| 7 | 1 | 6 | **6** | $[4, -1, 2, 1]$ |
| 8 | $-5$ | 1 | 6 | $[4, -1, 2, 1]$ |
| 9 | 4 | 5 | 6 | $[4, -1, 2, 1]$ |

Answer: 6, achieved by $[4, -1, 2, 1]$. The "restart or extend" choice at every step is the DP decision.

### 2.3.1 Application: Maximizing Stock Profit

Given prices $p[1..n]$, buy once and sell later ($i < j$) to maximize $p[j] - p[i]$ (a trade is mandatory).

**Direct D&C:** split at $m$. The optimal pair is either (1) entirely in the left half, (2) entirely in the right half, or (3) **crossing**: buy at the minimum price in $p[l..m]$ and sell at the maximum price in $p[m+1..r]$, found by two linear scans. Same recurrence $2T(n/2) + \Theta(n) = \Theta(n \log n)$.

**Reduction to Maximum Subarray ($O(n)$ preprocessing)**

Build the difference array of consecutive-day changes:

$$\text{Profit}[i] = p[i+1] - p[i].$$

Every interval telescopes:

$$\sum_{t=i}^{j-1} \text{Profit}[t] = \big(p[i+1] - p[i]\big) + \big(p[i+2] - p[i+1]\big) + \cdots + \big(p[j] - p[j-1]\big) = p[j] - p[i].$$

So the maximum-profit trade is exactly the maximum subarray of `Profit`, solvable by Kadane in $\Theta(n)$.

**Example:** prices $[3, 6, 10, 1, 8, 2, 6, 5, 10, 3, 1, 9]$ give differences

$$[3, 4, -9, 7, -6, 4, -1, 5, -7, -2, 8],$$

whose maximum subarray identifies the best buy/sell pair.

## 2.4 Multiplication and Master Theorem

### 2.4.1 Integer Multiplication (Karatsuba)

Given two $n$-bit integers $x$ and $y$, compute $x \cdot y$.

**Grade-School Multiplication**

The standard pencil-and-paper algorithm is $\Theta(n^2)$: multiply each bit of $y$ by $x$ (shift & add), giving $n$ rows of $O(n)$ bits each. Example: $163 \times 97$:

$$\begin{array}{rrr} & & 163 \\ \times & & 97 \\ \hline & 1141 & (163 \times 7) \\ + \quad 1467 & & (163 \times 9, \text{ shifted}) \\ \hline & & 15811 \end{array}$$

In binary, $10100011 \times 01100001$ similarly produces $n$ shifted partial products: $\Theta(n^2)$ single-bit multiplications and $\Theta(n^2)$ binary additions.

### 2.4.2 First D&C Attempt: Four Half-Size Products

Split $a = 2^{n/2}a_1 + a_0$, $b = 2^{n/2}b_1 + b_0$ (high/low $n/2$ bits):

$$a \cdot b = 2^n a_1 b_1 + 2^{n/2}(a_1 b_0 + a_0 b_1) + a_0 b_0.$$

Multiplication by $2^k$ is just a $k$-bit left shift (e.g. $12 = 00001100$, then $12 \cdot 2^3 = 96 = 01100000$), so the four products $a_1b_1, a_1b_0, a_0b_1, a_0b_0$ are computed recursively and combined with $\Theta(n)$ additions:

$$T(n) = 4T(n/2) + \Theta(n), \qquad T(1) = 1.$$

**Analysis: The Naive D&C Is Still $\Theta(n^2)$**

Expanding: $T(n) = 4^i T(n/2^i) + n(2^{i-1} + \cdots + 2 + 1)$; at $i = \log_2 n$,

$$T(n) = n^2 + n(2^{\log_2 n} - 1) = n^2 + n(n-1) = \Theta(n^2).$$

On the recursion tree, level $i$ costs $4^i \cdot n/2^i = n2^i$, so costs double each level and the leaves dominate ($4^{\log_2 n} = n^2$). Essentially the method still multiplies every bit of $a$ by every bit of $b$. The contrast with mergesort: one problem spawns **4** subproblems of size $n/2$, not 2.

**Karatsuba Split**

Write $x = a \cdot 2^{n/2} + b$, $y = c \cdot 2^{n/2} + d$ (split into high/low halves). Then:

$$x \cdot y = ac \cdot 2^n \; + \; \big[(a+b)(c+d) - ac - bd\big] \cdot 2^{n/2} \; + \; bd.$$

Instead of 4 half-size multiplications $(ac, ad, bc, bd)$, we use **3**: $ac$, $bd$, and $(a+b)(c+d)$, subtracting to recover $ad + bc$.

**Algorithm 14** Karatsuba Multiplication

```
function MULTIPLY(x, y, n)
    if n = 1 then
        return x · y
    end if
    a ← high n/2 bits of x;    b ← low n/2 bits of x
    c ← high n/2 bits of y;    d ← low n/2 bits of y
    ac ← MULTIPLY(a, c, n/2)
    bd ← MULTIPLY(b, d, n/2)
    ab_cd ← MULTIPLY(a + b, c + d, n/2)
    return ac ≪ n + (ab_cd − ac − bd) ≪ (n/2) + bd
end function
```

**Karatsuba Recurrence**

$$T(n) = 3T\left(\frac{n}{2}\right) + \Theta(n).$$

By Master Theorem case 1 ($\log_b a = \log_2 3 \approx 1.585 > 1$):

$$T(n) = \Theta(n^{\log_2 3}) \approx \Theta(n^{1.585}),$$

asymptotically better than $\Theta(n^2)$.

**Decimal illustration ($n = 2$ digits, no recursion needed).** Multiply $23 \times 14$: $a_1 = 2, a_0 = 3$, $c_1 = 1, d_0 = 4$.

$$ac = 2 \cdot 1 = 2, \qquad bd = 3 \cdot 4 = 12,$$
$$(a+b)(c+d) = 5 \cdot 5 = 25, \qquad ad + bc = 25 - 2 - 12 = 11,$$
$$23 \cdot 14 = 2 \cdot 100 + 11 \cdot 10 + 12 = 200 + 110 + 12 = \mathbf{322}.$$

Only **3** small multiplications $(2, 12, 25)$ instead of 4.

**History and Practice**

Karatsuba (1960) was the first algorithm to beat $\Theta(n^2)$. Splitting into $r$ parts and using fewer than $r^2$ products gives the Toom–Cook family: Toom-3 $\Rightarrow \Theta(n^{\log_3 5}) = \Theta(n^{1.465})$. FFT-based methods reached $\Theta(n \log n \log\log n)$; a $\Theta(n \log n)$ integer-multiplication algorithm was proven by Harvey & van der Hoeven in 2019. Real big-integer libraries combine methods by input size: textbook for tiny inputs, Karatsuba/Toom for medium, FFT for very large.

### 2.4.3 Master Theorem for Recurrences

For recurrences of the form:

$$T(n) = a\,T\left(\frac{n}{b}\right) + f(n), \quad a \geq 1,\ b > 1.$$

Compare $f(n)$ with $n^{\log_b a}$ (the "leaf work").

**Master Theorem — Three Cases**

1. **Case 1 (Leaves dominate):** If $f(n) = O(n^{\log_b a - \epsilon})$ for some $\epsilon > 0$,
$$T(n) = \Theta(n^{\log_b a}).$$
2. **Case 2 (Balanced):** If $f(n) = \Theta(n^{\log_b a} \log^k n)$ for some $k \geq 0$,
$$T(n) = \Theta(n^{\log_b a} \log^{k+1} n).$$
3. **Case 3 (Root dominates):** If $f(n) = \Omega(n^{\log_b a + \epsilon})$ and the **regularity condition** $a\,f(n/b) \leq c\,f(n)$ holds for some $c < 1$ and large $n$,
$$T(n) = \Theta(f(n)).$$

**Common D&C Recurrences (Memorize!)**

| Algorithm | Recurrence | Solution |
|---|---|---|
| Binary search | $T(n) = T(n/2) + \Theta(1)$ | $\Theta(\log n)$ (Case 2, $k = 0$) |
| Mergesort | $T(n) = 2T(n/2) + \Theta(n)$ | $\Theta(n \log n)$ (Case 2, $k = 0$) |
| Karatsuba | $T(n) = 3T(n/2) + \Theta(n)$ | $\Theta(n^{\log_2 3})$ (Case 1) |
| Strassen matrix | $T(n) = 7T(n/2) + \Theta(n^2)$ | $\Theta(n^{\log_2 7})$ (Case 1) |
| Bad D&C (naive 4-mult) | $T(n) = 4T(n/2) + \Theta(n)$ | $\Theta(n^2)$ (Case 1) |

**Why the Master Theorem Works (Recursion-Tree Intuition)**

For the simplified form $T(n) = aT(n/b)+n$: level $i$ contains $a^i$ subproblems of size $n/b^i$ and costs $n(a/b)^i$; there are $\log_b n$ levels and the leaves alone contribute $a^{\log_b n} = n^{\log_b a}$. Total:

$$T(n) = n^{\log_b a} + n \sum_{i=0}^{\log_b n - 1} \left(\frac{a}{b}\right)^i.$$

The geometric ratio $a/b$ decides everything:

- $a > b$: increasing geometric series, **leaves dominate**, $T(n) = \Theta(n^{\log_b a})$.
- $a = b$: every term is 1, each level costs $n$, $T(n) = \Theta(n \log n)$.
- $a < b$: decreasing geometric series is $\Theta(1)$, **the root dominates**, $T(n) = \Theta(n)$.

## 2.5 Matrix Multiplication and Strassen's Algorithm

Given two $n \times n$ matrices, compute $C = AB$ with $c_{ij} = \sum_{k=1}^{n} a_{ik} b_{kj}$. The brute-force triple loop costs $\Theta(n^3)$ ($n^2$ entries, each a length-$n$ dot product).

**Naive D&C:** partition each matrix into four $n/2 \times n/2$ blocks:

$$\begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix} = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix},$$

requiring **8** block multiplications (e.g. $C_{11} = A_{11}B_{11} + A_{12}B_{21}$) and $\Theta(n^2)$ block additions:

$$T(n) = 8T(n/2) + \Theta(n^2) = \Theta(n^3)$$

(Master case 1: $\log_2 8 = 3$, and $n^{\log_2 8} = n^3$ dominates $n^2$). No improvement.

**Strassen's Idea: 7 Block Products**

Strassen (1969) computes the $2 \times 2$ block product with only **7** recursive multiplications plus $\Theta(n^2)$ additions/subtractions:

$$\begin{aligned} P_1 &= A_{11}(B_{12} - B_{22}), & P_2 &= (A_{11} + A_{12})B_{22}, & P_3 &= (A_{21} + A_{22})B_{11}, \\ P_4 &= A_{22}(B_{21} - B_{11}), & P_5 &= (A_{11} + A_{22})(B_{11} + B_{22}), & & \\ P_6 &= (A_{12} - A_{22})(B_{21} + B_{22}), & P_7 &= (A_{11} - A_{21})(B_{11} + B_{12}), & & \end{aligned}$$

then

$$\begin{aligned} C_{11} &= P_5 + P_4 - P_2 + P_6, & C_{12} &= P_1 + P_2, \\ C_{21} &= P_3 + P_4, & C_{22} &= P_5 + P_1 - P_3 - P_7. \end{aligned}$$

Hence

$$T(n) = 7T(n/2) + \Theta(n^2) = \Theta(n^{\log_2 7}) = \Theta(n^{2.807\ldots}),$$

the first algorithm to beat cubic matrix multiplication. It is practical only for large dense matrices ($n \gtrsim 100$): the extra additions and recursion overhead make it slower on small matrices.

# 3 Randomized Algorithms

## 3.1 Why Randomization?

A randomized algorithm flips coins during execution. Worst-case analysis becomes **expected**-case analysis, often beating deterministic worst-case bounds.

**Two Types of Randomized Algorithms**

- **Las Vegas:** always correct; running time is a random variable (e.g. randomized Quicksort). We bound the expected runtime.
- **Monte Carlo:** always fast; has a small probability of being wrong (e.g. randomized min-cut / Karger). Repeat to reduce error.

**How We Measure Randomized Running Time**

We take the **expected running time on the worst-case input**: $\max_{I:|I|=n} \mathbb{E}_R[T(I,R)]$, where the expectation is only over the algorithm's internal randomness $R$ (random pivots, random permutations, ...). Randomized algorithms are often simpler and faster in practice; whether randomization can give an asymptotic advantage over deterministic algorithms remains open for some problems (e.g. $O(1)$-time hash tables with $O(n)$ space).

## 3.2 Probability Review

For a discrete random variable $X$ taking values with probabilities $\Pr[X = i]$:

$$\mathbb{E}[X] = \sum_i i \cdot \Pr[X = i].$$

**Dice warm-ups:**

- One fair 6-sided die: $\mathbb{E}[X] = \frac{1+2+\cdots+6}{6} = 3.5$.
- Sum of two dice: by linearity of expectation, $\mathbb{E}[X_1 + X_2] = 3.5 + 3.5 = \mathbf{7}$.
- Maximum of two dice: $\Pr[\max = k] = (k^2 - (k-1)^2)/36 = (2k-1)/36$, so

$$\mathbb{E}[\max] = 1 \cdot \tfrac{1}{36} + 2 \cdot \tfrac{3}{36} + 3 \cdot \tfrac{5}{36} + \cdots + 6 \cdot \tfrac{11}{36} \approx 4.47.$$

Two useful facts: $\mathbb{E}[XY] = \mathbb{E}[X]\mathbb{E}[Y]$ only when $X, Y$ are **independent** (but linearity of expectation needs no independence — that is why it is so powerful). For a non-negative integer-valued variable:

$$\mathbb{E}[X] = \sum_{i \geq 1} \Pr[X \geq i]$$

(the "tail-sum" formula, handy for waiting times).

## 3.3 Linearity of Expectation

For any random variables $X_1, X_2, \ldots, X_n$ (they need **not** be independent):

$$\mathbb{E}\left[\sum_{i=1}^{n} X_i\right] = \sum_{i=1}^{n} \mathbb{E}[X_i].$$

This is the single most useful tool for analyzing randomized algorithms.

**Indicator Variables**

To count "how many events happened", set:

$$X_i = \mathbf{1}\{\text{event } i \text{ occurs}\} = \begin{cases} 1 & \text{if event } i \text{ happens,} \\ 0 & \text{otherwise.} \end{cases}$$

Then $\mathbb{E}[X_i] = \Pr(\text{event } i)$, and $X = \sum X_i$ gives the total count with $\mathbb{E}[X] = \sum \Pr(\text{event } i)$.

## 3.4 Example: Hiring Problem

Interview candidates in random order; hire whenever candidate is better than all previous. How many hires (expected)?

Let $X_i = 1$ if candidate $i$ is hired (i.e. best among first $i$ candidates). In random order this occurs with probability $1/i$.

$$\mathbb{E}[X] = \sum_{i=1}^{n} \mathbb{E}[X_i] = \sum_{i=1}^{n} \frac{1}{i} = H_n = \Theta(\log n).$$

So the expected number of hires is the $n$-th harmonic number, $\ln n + O(1)$.

## 3.5 More Indicator-Variable Examples

### 3.5.1 Guessing Shuffled Cards

Shuffle $n$ cards and guess each card as it is revealed.

- **No memory:** each turn guess a uniformly random card from the full deck. Let $X_i = \mathbf{1}\{\text{guess } i \text{ correct}\}$; $\mathbb{E}[X_i] = 1/n$ regardless of other guesses, so $\mathbb{E}[X] = \sum_i 1/n = \mathbf{1}$.
- **With memory:** guess uniformly among the unseen cards. At turn $i$ there are $n-i+1$ candidates, so $\mathbb{E}[X_i] = 1/(n-i+1)$ and

$$\mathbb{E}[X] = \frac{1}{n} + \frac{1}{n-1} + \cdots + \frac{1}{1} = H_n = \Theta(\log n).$$

Note that the events are dependent in both games — linearity of expectation applies anyway.

### 3.5.2 Hash Table with Chaining

A hash function $h : U \to \{0, \ldots, m-1\}$ maps $n$ keys into a table of $m$ slots; collisions are resolved by a linked list per slot. Under the **simple uniform hashing assumption** ($h(x)$ uniform and mutually independent across keys), define $I_y = \mathbf{1}\{h(y) = h(x)\}$ for each stored key $y \neq x$. Then $\mathbb{E}[I_y] = 1/m$ and

$$\mathbb{E}[\text{keys colliding with } x] = \sum_{y \neq x} \mathbb{E}[I_y] \leq \frac{n}{m}.$$

With load factor $\alpha = n/m$, insert/delete/query take $O(1+\alpha)$ expected time; choosing $m = \Theta(n)$ makes this $O(1)$. (This is the idea behind C++ `unordered_map`.)

### 3.5.3 Birthday Paradox

With $k$ people and $n = 365$ equiprobable birthdays, let $X_{ij} = \mathbf{1}\{i, j \text{ share a birthday}\}$ ($\mathbb{E}[X_{ij}] = 1/n$):

$$\mathbb{E}[X] = \binom{k}{2}\frac{1}{n} = \frac{k(k-1)}{2n}.$$

Expecting $\geq 1$ collision means $k(k-1)/(2n) \geq 1$, i.e. $k \gtrsim \sqrt{2n} + 1$ — about **28** people for birthdays, far below the intuitive $365/2$. (This threshold also dictates when an $m$-slot hash table starts seeing collisions.)

## 3.6 Generating a Uniform Random Permutation

The hiring analysis assumes a uniformly random candidate order; the Fisher–Yates shuffle produces one in place:

**Algorithm 15** Fisher–Yates Shuffle

```
function RandomPermute(A)
    for i = 1 to n do
        swap A[i] ↔ A[Random(1, i)]
    end for
end function
```

$O(n)$ time and $O(1)$ extra space.

**Correctness: Every Permutation Has Probability $1/n!$**

Induct on $i$: after iteration $i$, every permutation of $A[1..i]$ occurs with probability $1/i!$. For target $(a_1, \ldots, a_i)$, let $j$ be the position of the original $A[i]$ in this target. The target appears iff (i) after iteration $i-1$ the other $i-1$ elements had the required order (probability $1/(i-1)!$ by induction), and (ii) $\text{Random}(1, i) = j$ (probability $1/i$). Multiplying: $1/(i-1)! \cdot 1/i = 1/i!$.

## 3.7 Geometric Distribution and Coupon Collector

**Waiting for a success.** If each coin flip succeeds with probability $p$, the number $X$ of flips to the first head is geometric:

$$\Pr[X = j] = (1-p)^{j-1}p, \qquad \mathbb{E}[X] = \sum_{i \geq 1}\Pr[X \geq i] = \sum_{i \geq 1}(1-p)^{i-1} = \frac{1}{p}.$$

**Coupon collector.** Each cereal box contains one of $n$ coupon types uniformly at random; how many boxes until all types are seen? Split the process into stages: after collecting $i$ distinct coupons, the chance the next box is new is $(n-i)/n$, so stage $i$ is geometric with mean $n/(n-i)$:

$$\mathbb{E}[X] = \sum_{i=0}^{n-1}\frac{n}{n-i} = n\sum_{j=1}^{n}\frac{1}{j} = nH_n = \Theta(n \log n).$$

## 3.8 Case Study: The Majority Element

Find the element appearing $> n/2$ times, assuming it exists.

| Method | Time | Space | Idea |
|---|---|---|---|
| Sort and count | $O(n \log n)$ | $O(1)$ | majority, if present, occupies the middle position |
| Divide and conquer | $O(n \log n)$ | $O(\log n)$ | winners of halves, equality tests only |
| Hashing | $O(n)$ expected | $O(n)$ | frequency map |
| Try-and-check | $O(n)$ expected | $O(1)$ | sample a random element, verify by counting; retry |
| Boyer–Moore vote | $O(n)$ worst-case | $O(1)$ | one cancellation pass, equality tests only |

**Algorithm 16** Boyer–Moore Majority Vote

```
c ← 0
for i = 1 to n do
    if c = 0 then
        m ← A[i];   c ← 1
    else if m = A[i] then
        c ← c + 1
    else
        c ← c − 1                    ▷ cancel A[i] against one stored copy of m
    end if
end for
return m
```

## Why the Vote Works

**Invariant:** after processing $A[1..i]$, the prefix equals "$c$ copies of $m$ plus elements already cancelled in pairs." Each cancellation removes two different items, so any single value can lose at most $n/2$ copies. A true majority ($> n/2$ copies) can never be completely cancelled — it must survive as $m$. (If a majority is not guaranteed, a second $O(n)$ verification pass is required.)

# 4 Quicksort

## 4.1 Deterministic Quicksort (Lomuto Partition)

Pick $A[r]$ as pivot; partition into $<$ pivot (left), $=$ pivot (middle), $>$ pivot (right).

**Algorithm 17** Quicksort

```
function QUICKSORT(A, p, r)
    if p < r then
        q ← PARTITION(A, p, r)
        QUICKSORT(A, p, q − 1)
        QUICKSORT(A, q + 1, r)
    end if
end function

function PARTITION(A, p, r)
    pivot ← A[r]
    i ← p − 1
    for j = p to r − 1 do
        if A[j] ≤ pivot then
            i ← i + 1
            swap A[i] ↔ A[j]
        end if
    end for
    swap A[i + 1] ↔ A[r]
    return i + 1
end function
```

**Partition Invariants**

At the end of each $j$-iteration in Partition:

- $A[p \dots i] \le pivot$,
- $A[i+1 \dots j] > pivot$,
- $A[j+1 \dots r-1]$ are unprocessed,
- $A[r] = pivot$.

Maintain with swaps; runtime is $\Theta(r - p + 1)$ on subarray size $n$.

**Worked example:** Partition $[2, 8, 7, 1, 3, 5, 6, 4]$ with pivot $A[r] = 4$ (Lomuto; bold marks $A[i]$ boundary):

| $j$ | $A[j] \le 4$? | $i$ after step | array |
|---|---|---|---|
| start | — | $p-1$ | $[2, 8, 7, 1, 3, 5, 6, \mathbf{4}]$ |
| $p$ ($A[j]=2$) | yes | $p$ | $[\mathbf{2}, 8, 7, 1, 3, 5, 6, 4]$ |
| $p+1$ (8) | no | $p$ | $[2, 8, 7, 1, 3, 5, 6, 4]$ |
| $p+2$ (7) | no | $p$ | $[2, 8, 7, 1, 3, 5, 6, 4]$ |
| $p+3$ (1) | yes | $p+1$ | $[2, \mathbf{1}, 7, 8, 3, 5, 6, 4]$ |
| $p+4$ (3) | yes | $p+2$ | $[2, 1, \mathbf{3}, 8, 7, 5, 6, 4]$ |
| $p+5, p+6$ (5, 6) | no | $p+2$ | unchanged |
| final swap | — | $q = p+3$ | $[2, 1, 3, \mathbf{4}, 7, 5, 6, 8]$ |

Pivot lands at its final sorted position; quicksort then recurses on $[2, 1, 3]$ and $[7, 5, 6, 8]$ independently.

## 4.2 Quicksort Analysis

### Deterministic Quicksort Worst-Case

If the input is **already sorted** and pivot is always the **last** (or first) element:

$$T(n) = T(n-1) + \Theta(n) \quad \Longrightarrow \quad T(n) = \Theta(n^2).$$

Same as insertion sort! This is why we randomize.

**Best case:** perfectly balanced splits give $T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n)$.

### "Balanced Enough" Splits: 1/4–3/4

Suppose every split were a **worst balanced case**, sizes $n/4$ and $3n/4$:

$$T(n) = T(n/4) + T(3n/4) + cn.$$

The recursion tree has a short left spine (depth $\log_4 n$) and a long right spine (depth $\log_{4/3} n$); every level does at most $cn$ work, so even here

$$T(n) \le cn \cdot \log_{4/3} n = O(n \log n).$$

With a random pivot, Pr[pivot rank lies in middle half $[n/4, 3n/4]$] $= 1/2$ — so on average every other split is good, which is already enough for $\Theta(n \log n)$ expected depth.

### Randomized Quicksort Expected Time

Pick pivot **uniformly at random** from $A[p..r]$ (swap it to position $r$ before Partition). Let $X$ = total number of **comparisons** made. Use indicator variables $X_{ij} = 1$ if elements of ranks $i$ and $j$ $(i < j)$ are ever compared.
Key insight: ranks $i$ and $j$ are compared iff **the first pivot chosen from within** $\{i, i+1, \ldots, j\}$ **is either** $i$ **or** $j$. That happens with probability $\dfrac{2}{j-i+1}$.

$$\mathbb{E}[X] = \sum_{1 \le i < j \le n} \frac{2}{j-i+1} = \sum_{k=2}^{n} \frac{2}{k} \cdot (n-k+1) = 2(n+1)H_n - 4n.$$

So $\mathbb{E}[X] = \Theta(n \log n)$, and therefore expected runtime is $\Theta(n \log n)$.

### Alternative View: Pivots Form a Random BST

Imagine inserting elements into a BST in the order in which they become pivots (the first pivot is the root, left-subarray pivots are its left subtree, ...). Two elements are compared exactly when one is an ancestor of the other — the same indicator argument, reinterpreted. The expected depth of the node ranked $i$ is

$$H_i + H_{n-i+1} - 2 \le 2H_n = O(\log n),$$

so the expected total comparisons are $\sum_i O(\log n) = O(n \log n)$. Equivalently, arrange the pair-comparison probabilities in an upper-triangular matrix: every row is a truncated harmonic series of $O(\log n)$ "large" terms $(2/k)$, and all $n$ rows together sum to $O(n \log n)$.

### Quicksort Properties

- **Stable:** no (Lomuto and Hoare partitions swap non-adjacent elements).
- **In-place:** yes (only $O(\log n)$ recursion stack; no auxiliary array).
- **Practical speed:** usually faster than Mergesort or Heapsort despite the same

$\Theta(n \log n)$ bound, thanks to better cache locality and low constants.

**Engineering Quicksort (What Real Libraries Do)**

- **Insertion-sort cutoff:** for subarrays of size $\lesssim 16$, switch to insertion sort, which beats recursive quicksort on tiny nearly-sorted ranges.
- **Median-of-three pivot:** choose the median of first/middle/last elements — cheaply defeats already-sorted and reverse inputs.
- **Introsort:** switch to Heapsort after recursion depth exceeds $2\log_2 n$, retaining Quicksort's average speed with a guaranteed $O(n \log n)$ worst case (used by C++ `std::sort`).
- **3-way partition (Dutch national flag):** group equal keys together; without it, arrays with many duplicate keys degenerate toward $\Theta(n^2)$.

## 4.3 Randomized Selection (Quickselect)

Find the $k$-th smallest element of an unsorted array (median is the $k = \lceil n/2 \rceil$ case). Reuse Partition: after partitioning on a random pivot, the pivot is at its final rank $k = m - l + 1$ inside the current segment; only **one** recursive call is needed.

**Algorithm 18** RandomizedSelect$(A, l, r, k)$ — return the $k$-th smallest of $A[l..r]$

```
if l = r then
    return A[l]
end if
pick pivot uniformly at random, swap it to r
m ← PARTITION(A, l, r)
pos ← m − l + 1                                  ▷ rank of pivot within A[l..r]
if k = pos then
    return A[m]
else if k < pos then
    return RANDOMIZEDSELECT(A, l, m − 1, k)
else
    return RANDOMIZEDSELECT(A, m + 1, r, k − pos)                ▷ relative rank
end if
```

**Worked example:** find the 4-th smallest of $[7, 2, 9, 1, 4, 8, 3]$ (sorted: $1, 2, 3, \mathbf{4}, 7, 8, 9$).

| Segment | $k$ | pivot | result |
|---|---|---|---|
| $[7, 2, 9, 1, 4, 8, 3]$ | 4 | 3 | partition $\to [2, 1, 3 \mid 7, 4, 8, 9]$, pivot rank $3 < 4$, go right |
| $[7, 4, 8, 9]$ | 1 | 9 | rank $4 > 1$, go left |
| $[7, 4, 8]$ | 1 | 8 | rank $3 > 1$, go left |
| $[7, 4]$ | 1 | 4 | partition $\to [4 \mid 7]$, rank $1 = k$, **return** 4 |

**Expected Time $\Theta(n)$**

Call a pivot **good** when its rank lies in the middle half $[n/4, 3n/4]$; this happens with probability $1/2$. After a good pivot the surviving subproblem has size $\leq 3n/4$; bad pivots only add linear partitioning cost. Stages between good pivots are geometric with expected count 2, so the expected total work is

$$2cn + 2c\tfrac{3}{4}n + 2c\left(\tfrac{3}{4}\right)^2 n + \cdots = 2cn \sum_{i \geq 0} \left(\tfrac{3}{4}\right)^i = 8cn = \Theta(n).$$

Worst case remains $\Theta(n^2)$ (probability tiny); the deterministic **median-of-medians**

algorithm achieves worst-case $O(n)$ by splitting into groups of 5, whose pivot is guaranteed to have at least $3n/10$ elements on each side: $T(n) = T(n/5)+T(7n/10)+\Theta(n) = O(n)$.

**Bonus: $k$-th of Two Sorted Arrays in $\Theta(\log k)$**

Given sorted arrays $A, B$ and rank $k$, compare $A[\lfloor k/2 \rfloor]$ and $B[\lfloor k/2 \rfloor]$ and discard the smaller half (none of its elements can be the $k$-th), then recurse with rank $k - \lfloor k/2 \rfloor$. Every step halves $k$; binary-search variants achieve $O(\log \min(m, n))$.

## 4.4 Hoare's Partition (Alternative)

Two pointers moving toward each other from opposite ends; swap inversions. Does slightly fewer swaps than Lomuto. Pivot is not guaranteed to end at index $q$; recur on $A[p..q]$ and $A[q+1..r]$, **skipping** $q$.

# 5 Heapsort

## 5.1 Priority Queue Motivation

A **priority queue** stores $n$ elements, each with a priority (key), and supports `Insert` and `Extract-Max` (min version: negate priorities). Concrete uses: OS process scheduling (the `top` table — new/preempted processes are inserted; when the CPU is free, the lowest-PR process is extracted), event-driven simulation, Dijkstra's shortest paths, and streaming top-$k$.

| Implementation | Insert | Extract-Max | Notes |
|---|---|---|---|
| Unsorted array/list | $O(1)$ | $O(n)$ | extract must scan everything |
| Sorted array/list | $O(n)$ | $O(1)$ | insert requires a shift |
| Balanced BST | $O(\log n)$ | $O(\log n)$ | correct bound but large pointer/balance overhead |
| **Binary heap** | $O(\log n)$ | $O(\log n)$ | compact array, no pointers, small constants |

## 5.2 Heap Data Structure

A (binary) **max-heap** is a complete binary tree satisfying:

$$A[\text{Parent}(i)] \geq A[i] \quad \forall i.$$

(Min-heap: $\leq$ instead.)

**Array Representation of a Heap**

Given 1-indexed array $A[1..n]$:

$$\begin{aligned} \text{Parent}(i) &= \lfloor i/2 \rfloor, \\ \text{Left}(i) &= 2i, \\ \text{Right}(i) &= 2i+1. \end{aligned}$$

A complete binary tree on $n$ nodes has height $\lfloor \lg n \rfloor$.

**Height derivation.** A complete tree of height $h$ has all levels $0, \ldots, h-1$ full ($2^h - 1$ nodes) and at least one node on level $h$; hence

$$2^0 + 2^1 + \cdots + 2^{h-1} = 2^h - 1 < n \quad \Longrightarrow \quad n \leq 2^0 + 2^1 + \cdots + 2^h = 2^{h+1} - 1,$$

so $2^h \leq n < 2^{h+1}$ and $h = \lfloor \lg n \rfloor$. This $\Theta(\log n)$ height is why every heap operation is fast.

## 5.3 Max-Heapify (Bubble-Down)

Assume children of node $i$ are heaps, but $A[i]$ may be too small. Fix by swapping $A[i]$ with the **larger** child, recursing down.

**Algorithm 19** Max-Heapify

```
function MaxHeapify(A, i, heap_size)
    L ← 2i;    R ← 2i + 1
    largest ← i
    if L ≤ heap_size and A[L] > A[largest] then
        largest ← L
    end if
    if R ≤ heap_size and A[R] > A[largest] then
        largest ← R
    end if
    if largest ≠ i then
        swap A[i] ↔ A[largest]
        MaxHeapify(A, largest, heap_size)
    end if
end function
```

Time: $O(\log n)$ per call (proportional to tree height).

## 5.4 Build Max-Heap

Naive approach: run Max-Heapify top-down from the **last non-leaf** ($\lfloor n/2 \rfloor$) *down to* root 1. Leaves are trivially heaps of size 1.

**Algorithm 20** Build Max-Heap

```
function BuildMaxHeap(A, n)
    for i = ⌊n/2⌋ downto 1 do
        MaxHeapify(A, i, n)
    end for
end function
```

**Build-Max-Heap Cost is $O(n)$, Not $O(n \log n)$**

Height-$h$ nodes are $\leq n/2^{h+1}$; each MaxHeapify on a height-$h$ node costs $O(h)$:

$$\sum_{h=0}^{\lfloor \lg n \rfloor} \frac{n}{2^{h+1}} \cdot O(h) = O\left( n \sum_{h=0}^{\infty} \frac{h}{2^h} \right) = O(2n) = O(n).$$

(Building from the leaves up is much cheaper than building from the root down.)

**Per-level bookkeeping** (take $n = 2^{h+1} - 1$, a perfectly full tree):

| Level (root = 0) | # nodes | heapify cost per node | total for level |
|---|---|---|---|
| 0 | 1 | $h$ | $h$ |
| 1 | 2 | $h-1$ | $2(h-1)$ |
| 2 | 4 | $h-2$ | $4(h-2)$ |
| $\vdots$ | $\vdots$ | $\vdots$ | $\vdots$ |
| $i$ | $2^i$ | $h-i$ | $2^i(h-i)$ |
| $h$ | $2^h$ | 0 | 0 |

Summing with $j = h - i$:

$$\sum_{i=0}^{h-1} 2^i (h-i) = 2^h \sum_{j=1}^{h} \frac{j}{2^j} = 2^h \left( 2 - \frac{h+2}{2^h} \right) = 2^{h+1} - h - 2 < n,$$

using $\sum_{j=1}^{h} j/2^j = 2 - (h+2)/2^h$. Nearly all the work is at the bottom levels; the root level costs only $h$.

## 5.5 Heapsort Algorithm

(1) Build max-heap on $A[1..n]$. (2) Repeatedly: swap max (root $A[1]$) with last element, shrink heap, MaxHeapify root.

**Algorithm 21** Heapsort

```
function HEAPSORT(A, n)
    BUILDMAXHEAP(A, n)
    for i = n downto 2 do
        swap A[1] ↔ A[i]
        heap_size ← i − 1
        MAXHEAPIFY(A, 1, heap_size)
    end for
end function
```

**Heapsort Analysis**

- Build: $O(n)$.
- $n-1$ swaps each followed by MaxHeapify: $(n-1)\,O(\log n) = O(n \log n)$.
- **Total:** $O(n \log n)$ (best = worst = average, unlike Quicksort).
- **Stable?** No.
- **In-place?** Yes (only $O(1)$ extra storage, no recursion stack if MaxHeapify is written iteratively).

## 5.6 Priority Queue Operations (Applications of Heaps)

Max-priority queue supports: `Maximum`, `ExtractMax`, `IncreaseKey`, `Insert`.

**Algorithm 22** Priority Queue Operations

```
function EXTRACTMAX(A, heap_size)
    max ← A[1]
    A[1] ← A[heap_size];   heap_size ← heap_size − 1
    MAXHEAPIFY(A, 1, heap_size)
    return max
end function

function INCREASEKEY(A, i, key)
    if key < A[i] then error
    end if
    A[i] ← key
    while i > 1 and A[Parent(i)] < A[i] do                ▷ bubble up
        swap A[i] ↔ A[Parent(i)]
        i ← Parent(i)
    end while
end function

function INSERT(A, key, heap_size)
    heap_size ← heap_size + 1
    A[heap_size] ← −∞
    INCREASEKEY(A, heap_size, key)
end function
```

Each operation runs in $O(\log n)$ time.

**Extract-Max trace** on $[42, 29, 18, 14, 7, 6]$:

| | | |
|---|---|---|
| initial | save $m = 42$, move 6 to root | $[\mathbf{6}, 29, 18, 14, 7]$ |
| bubble down | $6 < 29$, swap with larger child | $[\mathbf{29}, 6, 18, 14, 7]$ |
| bubble down | $6 < \max(14, 7) = 14$, swap with 14 | $[29, \mathbf{14}, 18, 6, 7]$ |

Returned 42; heap is now $[29, 14, 18, 6, 7]$.

**Insert trace (bubble up):** insert 20 into $[29, 14, 18, 6, 7]$:

| | | |
|---|---|---|
| append | $[29, 14, 18, 6, 7, \mathbf{20}]$ | parent of 20 is 18 |
| swap up | $20 > 18$ | $[29, 14, \mathbf{20}, 6, 7, 18]$ |
| stop | parent is $29 > 20$ | done |

## 5.7 Applications

### 5.7.1 Merging $k$ Sorted Arrays

Combine $k$ sorted arrays of length $n$ into one sorted sequence of length $kn$.

| Solution | Time | Idea |
|---|---|---|
| Sort everything | $O(kn \log(kn))$ | ignore existing order |
| Divide & conquer | $O(kn \log k)$ | pairwise 2-way merges, $\log_2 k$ rounds of $kn$ work |
| Min-heap | $O(kn \log k)$ | keep one current head per array in a heap of size $k$ |

**Heap method:** insert the first element of each array (tagged with its source array) into a min-heap; repeatedly extract-min and insert the next element from the same source array. Each of the $kn$ elements costs one insert + one extract, each $O(\log k)$. Uses only $O(k)$ extra space, and it works for online/streaming inputs and huge files (external merge sort).

### 5.7.2 Delete and Update-Priority

Store `(key, priority, payload)` tuples, where `key` uniquely identifies the record.

- `Delete(key)`: locate its heap index $k$, swap $A[k]$ with $A[n]$, shrink, then **bubble up or bubble down** from $k$ (either direction may be needed) — $O(\log n)$ once located.
- `UpdatePriority(key, p)`: set the new priority and bubble in the appropriate direction.

Locating an element by key requires a separate **position map** (hash table: key $\to$ heap index), updated on every swap. It preserves the $O(\log n)$ bounds but adds real overhead, which is why C++/Python standard heaps do not support it directly (`std::set` or lazy deletion are common workarounds).

### 5.7.3 The Skyline Problem

Given buildings $(x_L, x_R, y)$, output the skyline key points (positions where the maximum height changes).

- **D&C solution:** recursively get the skylines of two halves; merge them like merge-sort while tracking current heights $h_1, h_2$ of each side. At each next boundary $x = \min(x_1, x_2)$, advance that side and append $(x, \max(h_1, h_2))$ only when it differs from the last emitted height. An $O(n)$ merge gives $T(n) = 2T(n/2) + \Theta(n) = O(n \log n)$.
- **Heap (sweep-line) solution:** sort all vertical boundaries $(x, y, \text{id}, L/R)$; sweep left to right, inserting a building at its left edge and deleting it at its right edge from a max-heap of live heights. The heap top is the current skyline height; emit a key point whenever it changes. $O(n \log n)$.

# 6 Linear-Time Sorting

All comparison-based sorts have *worst-case* $\Omega(n \log n)$ lower bound (decision-tree height). The following beat the bound by **not comparing keys directly**.

**When Can We Beat $\Omega(n \log n)$?**

We need extra information about the keys: their values lie in a **small integer range**, or they have structure that allows *digit-wise* processing.

## 6.1 The Decision-Tree Model and the Sorting Lower Bound

A **comparison-based** algorithm makes decisions only from pairwise comparisons ("$a_i \le a_j$?"); its behavior depends solely on the relative order, never on actual values. Merge, heap and quick sort are comparison-based; counting/radix/bucket sort are not.

Represent any execution as a **decision tree**: each internal node is one comparison with two outcomes (left/right branch); each leaf is one possible output. The **worst-case number of comparisons is the height** of the tree, and a binary tree with $N$ leaves has height at least $\lceil \log_2 N \rceil$.

**Warm-up: Lower Bound for Binary Search**

Searching a sorted array of size $n$ has $n+1$ distinct outputs — one per element, plus "not found." Hence the decision tree needs $\ge n+1$ leaves and height $\ge \log_2(n+1) = \Omega(\log n)$. Binary search matches this bound, so it is asymptotically optimal.

**Exercise: The Defective Coin Balance**

Among $n$ coins, at most one is too heavy or too light (unknown which); a three-way balance reports left-heavy / balanced / right-heavy.

- **Lower bound:** there are $2n+1$ outcomes ($n$ heavy, $n$ light, all genuine), so the ternary decision tree has $\ge 2n+1$ leaves: height $\ge \log_3(2n+1)$ weighings.
- **Algorithm:** split into three equal groups $C_1, C_2, C_3$. Two weighings ($C_1$ vs $C_2$, then $C_1$ vs $C_3$) distinguish the 7 outcomes $C_1H, C_1L, C_2H, C_2L, C_3H, C_3L$, "none"; recurse into the defective group:

$$T(n) = T(n/3) + 2 = 2\log_3 n = O(\log n),$$

matching the lower bound up to constants.

**Theorem: Sorting Requires $\Omega(n \log n)$ Comparisons**

A sorting algorithm must be able to output any of the $n!$ input permutations, so its decision tree has $\ge n!$ leaves and height

$$h \ge \log_2(n!) = \sum_{i=1}^{n} \log_2 i \ge \sum_{i=\lceil n/2 \rceil}^{n} \log_2 i \ge \frac{n}{2} \log_2 \frac{n}{2} = \Omega(n \log n)$$

(equivalently use Stirling: $n! \ge (n/e)^n$, so $\log_2 n! = \Theta(n \log n)$). Therefore **every** comparison-based sort needs $\Omega(n \log n)$ comparisons in the worst case — Merge Sort and Heapsort are thus asymptotically optimal among comparison sorts. To do better we must leave the comparison model.

**Algorithm vs problem lower bounds.** "Insertion sort is $\Omega(n^2)$" is an algorithm bound (a particular algorithm's cost); "sorting is $\Omega(n \log n)$" is a problem bound (no comparison-based algorithm can beat it, proved independently of how clever the algorithm is). Problem lower bounds are the rarer and more valuable kind — they tell us when to

stop improving and point toward new models (non-comparison methods, randomization, average-case assumptions).

## 6.2 Counting Sort

Assume each of $n$ input keys is an integer in $\{0, 1, \ldots, k\}$.

**Algorithm 23** Counting Sort

```
function CountingSort(A, B, n, k)
    for i = 0 to k do
        C[i] ← 0
    end for
    for j = 1 to n do
        C[A[j]] ← C[A[j]] + 1                          ▷ count frequencies
    end for
    for i = 1 to k do
        C[i] ← C[i] + C[i − 1]                         ▷ prefix sums = "how many ≤ i"
    end for
    for j = n downto 1 do
        B[C[A[j]]] ← A[j]
        C[A[j]] ← C[A[j]] − 1
    end for
end function
```

**Counting Sort Properties**

- Runtime: $\Theta(n + k)$. If $k = O(n)$ then $\Theta(n)$ overall.
- **Stable:** yes (fill $B$ *right-to-left* so equal keys preserve input order). This is critical for being used as a subroutine in Radix Sort.
- **In-place:** no (uses arrays $C$ of size $k + 1$ and output $B$ of size $n$).

**Worked example:** $A = [2, 5, 3, 0, 2, 3, 0, 3]$, $k = 5$.

| $i$ (key value) | 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|
| frequencies $C$ | 2 | 0 | 2 | 3 | 0 | 1 |
| prefix sums $C$ | 2 | 2 | 4 | 7 | 7 | 8 |

Placing right-to-left (equal keys keep their order): $B = [0, 0, 2, 2, 3, 3, 3, 5]$.

## 6.3 Radix Sort

Sort $d$-digit keys by applying a **stable** sort digit-by-digit, least significant digit first (LSD). Each digit sort is Counting Sort on that digit.

**Algorithm 24** Radix Sort

```
function RadixSort(A, d, n)
    for i = 1 to d do
        Use a stable sort to sort A on digit i (the i-th least significant)
    end for
end function
```

**Radix Sort Correctness & Complexity**

- Why LSD works: after pass $i$, numbers are sorted correctly w.r.t. their low $i$ digits;

next pass (higher digit) is stable so ties keep the lower-order ordering.
- Each digit in $\{0, \ldots, k-1\}$ sorted by Counting Sort in $\Theta(n+k)$.
- Total: $\Theta(d(n+k))$. If $d$ is constant and $k = O(n)$, this is $\Theta(n)$.
- **Stable:** yes (because inner sort is stable).

**Radix Sort Gotchas**

- Must process digits **from LSD to MSD** (not the other way around).
- Inner sort **must be stable**; otherwise equal-digit elements lose their previously-computed order.

**Correctness by Induction on Digit Position**

**Claim:** after pass $i$, keys are sorted by their low-order $i$ digits. Base $i = 0$ vacuous. Assume sorted by low $i-1$ digits; after stably sorting on digit $i$:
- two keys differing in digit $i$ are correctly ordered by that more significant digit;
- two keys equal in digit $i$ keep their pre-pass order by **stability**, which is correct ordering by the low $i-1$ digits (induction hypothesis).

Both cases give correctness by low $i$ digits; after $d$ passes the keys are fully sorted.

**Worked example (3 LSD passes):**

| | |
|---|---|
| input | $[329, 457, 657, 839, 436, 720, 355]$ |
| pass 1 (units) | $[720, 355, 436, 457, 657, 329, 839]$ |
| pass 2 (tens) | $[720, 329, 436, 839, 355, 457, 657]$ |
| pass 3 (hundreds) | $[329, 355, 436, 457, 657, 720, 839]$ |

**Bit-Parameter Analysis ($w$-bit Integers)**

Each "digit" spans $\log_2 k$ bits, so the number of passes is $d = \lceil w / \log k \rceil$ and total time is

$$\Theta\Big(\lceil \tfrac{w}{\log k} \rceil (n+k)\Big).$$

The $(n+k)$ factor is minimized at $k = n$ (equal split between pass count and bucket overhead), giving $\Theta(\lceil w/\log n \rceil n)$. In the standard word-RAM regime $w = O(\log n)$ (pointers/indices fit in a machine word; all machines from 8-bit to 64-bit satisfy this for realistic RAM sizes), radix sort runs in $\Theta(n)$. It also extends to fixed-length strings and to floating-point keys (their IEEE-754 bit order is monotone after a sign flip).

## 6.4 Bucket Sort

Assume input is drawn **uniformly at random** from $[0, 1)$. Distribute into $n$ equal-sized buckets, sort each bucket (e.g. insertion sort), concatenate.

**Algorithm 25** Bucket Sort

```
function BucketSort(A, n)
    Create B[0..n − 1] (each an empty list)
    for i = 1 to n do
        insert A[i] into B[⌊n · A[i]⌋]
    end for
    for i = 0 to n − 1 do
        sort list B[i] (e.g. insertion sort)
    end for
    Concatenate B[0], B[1], ..., B[n − 1] in order
end function
```

## Bucket Sort Expected Time

With uniform input, each bucket has $O(1)$ elements in expectation; sorting each by insertion sort is $O(1)$ per bucket.

$$\mathbb{E}[T(n)] = \Theta(n) + \sum_{i=0}^{n-1} O(1) = \Theta(n).$$

Worst case (all elements fall in one bucket): $\Theta(n^2)$ if inner sort is insertion sort.

## Why the Expected Cost Is Linear: $\mathbb{E}[X_0^2] = O(1)$

Let $X_i$ be the number of elements in bucket $i$ and $Y_j = \mathbf{1}\{A[j] \text{ lands in bucket } 0\}$, so $X_0 = \sum_{j=1}^{n} Y_j$. Insertion sort on bucket $i$ costs $\Theta(X_i^2)$; by identical distributions,

$$\mathbb{E}\left[\sum_{i=0}^{n-1} X_i^2\right] = n\,\mathbb{E}[X_0^2].$$

Expand the square and use independence of distinct $Y_j$ ($\Pr[Y_j = 1] = 1/n$):

$$\mathbb{E}[X_0^2] = \sum_j \mathbb{E}[Y_j^2] + 2\sum_{j<\ell} \mathbb{E}[Y_j Y_\ell] = n \cdot \frac{1}{n} + 2\binom{n}{2}\frac{1}{n^2} = 1 + 1 - \frac{1}{n} = 2 - \frac{1}{n} = O(1).$$

So bucket sorting costs $O(n)$ expected time total (plus the $O(n)$ distribution/concatenation passes). If inputs are not in $[0, 1)$, normalize first; with non-uniform distributions a custom bucket partition is needed.

## Sorting Algorithm Cheat Sheet

| Algorithm | Worst | Average | Space | Comparison? | Stable | Cache | Parallel |
|---|---|---|---|---|---|---|---|
| Insertion | $\Theta(n^2)$ | $\Theta(n^2)$ | $\Theta(1)$ | yes | yes | good | no |
| Mergesort | $\Theta(n \log n)$ | $\Theta(n \log n)$ | $\Theta(n)$ | yes | yes | good | good |
| Quicksort | $\Theta(n^2)$ | $\Theta(n \log n)$ | $\Theta(\log n)$ | yes | no | good | good |
| Heapsort | $\Theta(n \log n)$ | $\Theta(n \log n)$ | $\Theta(1)$ | yes | no | bad | no |
| Counting | $\Theta(n + k)$ | $\Theta(n + k)$ | $\Theta(n + k)$ | no | yes | bad | no |
| Radix | $\Theta(\lceil w/\log n \rceil n)$ | same | $\Theta(n)$ | no | yes | bad | good |
| Bucket | $\Theta(n^2)$ | $\Theta(n)$ expected | $\Theta(n)$ | no | yes | bad | good |

# 7 Greedy Algorithms

A **greedy algorithm** always makes the choice that looks best at the moment and adds it to the current partial solution. It builds a solution to an optimization problem through a sequence of choices that are:

- **Feasible:** satisfying the given constraints.
- **Locally optimal:** make the choice that looks best right now, without considering the future.
- **Irrevocable:** once a choice is made it is never undone (no backtracking).

**When Does Greedy Work?**

- A greedy algorithm is correct when the problem has the **greedy-choice property** (a locally optimal choice can be part of some global optimum) together with **optimal substructure** (an optimal solution contains optimal solutions to subproblems).
- When they work, greedy algorithms are usually the simplest and most efficient algorithms available. A sorting step often dominates the cost, giving $\Theta(n \log n)$.
- When they fail, they may still give fast **approximations**.

**Greedy Correctness Must Be Proven!**

"Picking the locally best option" is not automatically optimal. The correctness (optimality) of a greedy algorithm is usually non-obvious. Two standard proof techniques:

1. **Exchange argument:** transform any optimal solution into the greedy solution one choice at a time, without worsening it (Interval Scheduling, Fractional Knapsack, Huffman).
2. **Greedy stays ahead / match a lower bound:** show the greedy solution reaches a universal lower bound (Interval Partitioning vs. depth).

## 7.1 Warm-up: Change-Making

Input: amount $n$ and coin denominations $d_1 > d_2 > \cdots > d_m$. Output: the minimum number of coins summing to $n$.

**Greedy rule:** give as many coins as possible of the largest denomination, then the second largest, and so on.

**Example (Hong Kong coins):** $n = 19$, denominations $10, 5, 2, 1$.

$$19 = 10 + 5 + 2 + 2 \quad \Longrightarrow \quad 4 \text{ coins, and this is optimal.}$$

**Greedy Change-Making Is Not Always Optimal**

British coinage before 1971 used denominations $1, 3, 6, 12, 24, 30, 60$. For $n = 48$:

- Greedy: $48 = 30 + 12 + 6$ (3 coins).
- Optimal: $48 = 24 + 24$ (2 coins).

The greedy choice (take 30) "steals" capacity that two better coins could have used. For arbitrary denominations, change-making requires **dynamic programming**.

## 7.2 Warm-up: Max Sum of Non-Adjacent Elements

Given $n$ positive numbers $A[1], \ldots, A[n]$, pick a subset of **pairwise non-adjacent** elements with maximum total sum. Example: $A = (1, 8, 6, 3, 6) \Rightarrow$ pick $A[2], A[5]$ with sum $8 + 6 = 14$.

**Greedy attempt:** repeatedly take the largest remaining element, then delete it and its two neighbors.

**Algorithm 26** Greedy Non-Adjacent Selection (Not Optimal!)

$S \leftarrow \emptyset$
**while** some elements remain in $A$ **do**
  pick index $i$ with the largest remaining $A[i]$
  $S \leftarrow S \cup \{A[i]\}$
  mark $A[i-1], A[i], A[i+1]$ as deleted
**end while**
**return** $S$

**Counterexample:** $(1, 4, 6, 4)$

- Greedy takes the maximum 6, which forces deleting both neighbors 4 and 4, then takes 1: sum $= 6 + 1 = 7$.
- Optimal: take the two 4's (positions 2 and 4): sum $= 8 > 7$.

The optimal solution is given by DP: $\mathrm{OPT}(i) = \max\big(\mathrm{OPT}(i-1),\ \mathrm{OPT}(i-2) + A[i]\big)$.

## 7.3 Interval Scheduling

Job $j$ starts at $s_j$ and finishes at $f_j$. Two jobs are **compatible** if their intervals do not overlap. Goal: select a **maximum-size** subset of mutually compatible jobs.

### 7.3.1 Three Tempting Greedy Rules (All Fail)

**Counterexamples**

1. **Earliest start time.** One long job $e = [0, 10)$ starting first can block four short jobs $[1, 2), [2, 3), [3, 4), [4, 5)$: greedy chooses $\{e\}$ (size 1) instead of $\{a, b, c, d\}$ (size 4).
2. **Shortest interval.** Let $a = [0, 4)$, $b = [4, 8)$, and a very short $c = [3.5, 4.5)$ overlapping both. Shortest-first chooses $\{c\}$ (size 1) instead of $\{a, b\}$ (size 2).
3. **Fewest conflicts.** Count each job's overlapping jobs $c_j$ and schedule in ascending $c_j$. A job conflicting with only a few others can still sit in the middle of a long compatible chain and, once chosen, force several of them out — the slide example forces $\{a, f, d\}$ (size 3) instead of $\{a, b, c, d\}$ (size 4). The number of conflicts is a global statistic and does not measure local opportunity cost.

### 7.3.2 The Correct Rule: Earliest Finish Time

**Greedy algorithm:** consider jobs in increasing order of finish time; take a job whenever it is compatible with the jobs already chosen. Intuition: finishing early **leaves the maximum remaining time** for all later jobs.

**Algorithm 27** Interval Scheduling (Earliest Finish Time)

```
Sort jobs so that f_1 ≤ f_2 ≤ ··· ≤ f_n
A ← ∅;   last ← 0                          ▷ finish time of the last selected job
for j = 1 to n do
    if s_j ≥ last then                     ▷ compatible: ends are inclusive, so ≥ is fine
        A ← A ∪ {j};   last ← f_j
    end if
end for
return A
```

**Running time:** $\Theta(n \log n)$, dominated by sorting; the scan itself is $\Theta(n)$.

**Worked trace.** Jobs sorted by finish time:

$$j_1 = [0, 2),\ j_2 = [1, 4),\ j_3 = [3, 5),\ j_4 = [4, 6),\ j_5 = [5, 7).$$

- Select $j_1$ (earliest finish), set $last = 2$.
- Skip $j_2$: $s_2 = 1 < 2$, overlaps $j_1$.
- Select $j_3$: $s_3 = 3 \geq 2$, set $last = 5$.
- Skip $j_4$: $s_4 = 4 < 5$.
- Select $j_5$: $s_5 = 5 \geq 5$.

Greedy returns $\{j_1, j_3, j_5\}$, size 3 — the maximum possible.

### 7.3.3 Correctness Proof (Exchange Argument)

**Theorem: Earliest-Finish-Time Greedy Is Optimal**

Let $G = (i_1, i_2, \ldots, i_k)$ be the jobs selected by greedy and $O = (j_1, j_2, \ldots, j_m)$ be any optimal schedule, both in selection order.

1. Suppose $G \neq O$. Find the largest $r$ such that $i_1 = j_1, \ldots, i_r = j_r$ (the common prefix; $r = 0$ if they differ immediately).
2. By the greedy rule, $i_{r+1}$ is the compatible job with the **earliest finish**, so $f_{i_{r+1}} \leq f_{j_{r+1}}$.
3. Build $O^*$ from $O$ by replacing $j_{r+1}$ with $i_{r+1}$. Because $i_{r+1}$ finishes no later than $j_{r+1}$, it is still compatible with $j_{r+2}, \ldots, j_m$; hence $O^*$ is a feasible schedule with the same size as $O$, so $O^*$ is also optimal.
4. $O^*$ now shares the first $r + 1$ jobs with $G$. Repeat the replacement; the common prefix grows each time, so eventually $O$ is transformed into $G$ with its size (value) unchanged.

Therefore $|G| = |O|$, i.e. greedy is optimal. Equivalently, the greedy schedule "stays ahead": the $r$-th greedy job finishes no later than the $r$-th job of any schedule, so greedy can always fit at least as many jobs.

## 7.4 Exercise: Interval Covering

Given $n$ intervals that together cover $[0, 1]$; interval $x$ has start $x.s$ and finish $x.f$. Find the **minimum** number of intervals whose union covers $[0, 1]$. (Assume all endpoints are distinct.)

**Greedy idea:** maintain the rightmost covered point $x$. Among all intervals covering $x$ ($I.s \leq x < I.f$), pick the one extending **furthest to the right**; advance $x$ to its finish and repeat until $x \geq 1$.

**Algorithm 28** Greedy Interval Covering

$x \leftarrow 0$; $\quad G \leftarrow \emptyset$
**while** $x < 1$ **do**
  pick interval $I$ with $I.s \le x$ that maximizes $I.f$
  $G \leftarrow G \cup \{I\}$; $\quad x \leftarrow I.f$
**end while**
**return** $G$

**Worked trace.** Intervals $A = [0, 0.3]$, $B = [0.2, 0.6]$, $C = [0.5, 0.8]$, $D = [0.75, 1]$:

- $x = 0$: only $A$ covers 0 $\Rightarrow$ take $A$, $x = 0.3$.
- $x = 0.3$: $B$ covers 0.3 and extends furthest $\Rightarrow$ take $B$, $x = 0.6$.
- $x = 0.6$: $C$ covers 0.6 ($D$ starts at 0.75) $\Rightarrow$ take $C$, $x = 0.8$.
- $x = 0.8$: take $D$, $x = 1$. Done with 4 intervals.

**Optimality Sketch & Implementation**

Let $O = (o_1, \ldots, o_m)$ be optimal and $G = (g_1, \ldots, g_k)$ greedy, ordered by start time. Let $i$ be the first index with $o_i \neq g_i$. The greedy choice extends at least as far: $g_i.f \ge o_i.f$. Replacing $o_i$ by $g_i$ keeps the cover valid; iterating converts $O$ into $G$, so $k \le m$ and greedy is optimal. Sorting plus a single sweep (among intervals already started, track the maximum finish) gives $O(n \log n)$.

## 7.5 Fractional Knapsack

Input: $n$ items, item $i$ has weight $w_i$ and value $v_i$; knapsack capacity $W$. Choose fractions $0 \le x_i \le 1$ maximizing $\sum_i x_i v_i$ subject to $\sum_i x_i w_i \le W$.

- **0/1 knapsack:** $x_i \in \{0, 1\}$ (take the whole item or nothing) — greedy fails; needs DP.
- **Fractional knapsack:** $x_i \in [0, 1]$ (items are divisible, e.g. liquids, grain) — greedy is optimal.

**Greedy rule:** sort items by **value per unit weight** $v_i/w_i$ decreasing; take as much as possible of each item in that order. Running time $\Theta(n \log n)$.

**Algorithm 29** Fractional Knapsack (Greedy)

Sort items so that $\frac{v_1}{w_1} \ge \frac{v_2}{w_2} \ge \cdots \ge \frac{v_n}{w_n}$
$w \leftarrow W$ ▷ remaining capacity
**for** $i = 1$ **to** $n$ **do**
  **if** $w_i \le w$ **then**
    $x_i \leftarrow 1$; $\quad w \leftarrow w - w_i$
  **else**
    $x_i \leftarrow w/w_i$; $\quad w \leftarrow 0$; $\quad$ **return**
  **end if**
**end for**

**Worked example.** $W = 50$; items $(w_i, v_i) = (10, 60), (20, 100), (30, 120)$, with ratios $6, 5, 4$.

1. Take all of item 1: used weight 10, value 60.
2. Take all of item 2: used weight 30, value 160.
3. Fill the remaining 20 units with a fraction $20/30 = 2/3$ of item 3: added value $\frac{2}{3} \cdot 120 = 80$.

Optimal total: $60 + 100 + 80 = \mathbf{240}$, weight exactly 50.

**Why the Same Greedy Fails for 0/1 Knapsack**

On the example above, "take by ratio while the whole item fits" grabs items 1 and 2 (weight 30, value 160) and then item 3 does not fit — leaving 20 units of capacity wasted. The 0/1 optimum takes items 2 and 3: weight 50, value 220 $>$ 160. Being forced to leave capacity unused is exactly what divisibility (the fractional version) avoids.

**Correctness Proof (Exchange Argument)**

Assume $\sum_i w_i \geq W$ (otherwise take everything; trivially optimal), so both greedy $G = (x_1, \ldots, x_k, 0, \ldots, 0)$ (where $x_i = 1$ for $i < k$ and $0 \leq x_k \leq 1$) and any optimum $O = (y_1, \ldots, y_n)$ fill the knapsack exactly.

1. Let $i$ be the first index where $x_i \neq y_i$. By the greedy ordering and the prefix agreement, greedy takes at least as much of the best remaining item: $x_i \geq y_i$.
2. Increase $y_i$ to $x_i$ and compensate by removing total weight $(x_i - y_i)w_i$ from items $i+1, \ldots, n$ (enough such weight exists because both solutions pack exactly $W$).
3. Value cannot decrease: every item $j > i$ has ratio $v_j/w_j \leq v_i/w_i$, so swapping weight onto item $i$ replaces "cheaper" weight with "dearer" weight.
4. Since $O$ was optimal, its value also cannot increase, so it stays exactly the same: the modified $O$ is still optimal and now agrees with $G$ up to index $i+1$.

Repeating converts $O$ into $G$ with unchanged value $\Rightarrow$ greedy is optimal.

## 7.6 Interval Partitioning

Lecture $j$ occupies $[s_j, f_j)$. Goal: schedule all lectures using the **minimum number of classrooms** so that no two lectures overlap in the same room.

**Greedy algorithm:** sort lectures by increasing **start** time; assign each lecture to any classroom whose last lecture has finished; if none exists, open a new classroom.

**Algorithm 30** Interval Partitioning (Greedy by Start Time)

```
Sort lectures so that s_1 ≤ s_2 ≤ ··· ≤ s_n
H ← empty min-heap of classroom finish times
for j = 1 to n do
    if H ≠ ∅ and min(H) ≤ s_j then
        ExtractMin(H)                          ▷ reuse that classroom
    end if
    Insert(H, f_j)                             ▷ new room if none was reused
end for
return |H|                                     ▷ number of classrooms used
```

**Worked trace.** Lectures (sorted by start): $A = [1, 4)$, $B = [2, 6)$, $F = [3, 3.5)$, $C = [4, 5)$, $D = [5, 7)$, $E = [6, 8)$.

| Lecture | Decision | Heap of finish times | Overlapping rooms |
|---|---|---|---|
| $A[1, 4)$ | open room 1 | $\{4\}$ | 1 |
| $B[2, 6)$ | $4 > 2$, open room 2 | $\{4, 6\}$ | 2 |
| $F[3, 3.5)$ | $4 > 3$, open room 3 | $\{3.5, 4, 6\}$ | 3 |
| $C[4, 5)$ | $3.5 \leq 4$, reuse | $\{4, 5, 6\}$ | 3 |
| $D[5, 7)$ | $4 \leq 5$, reuse | $\{5, 6, 7\}$ | 3 |
| $E[6, 8)$ | $5 \leq 6$, reuse | $\{6, 7, 8\}$ | 3 |

Three rooms are used; at time 3 lectures $A, B, F$ overlap simultaneously, so 3 is unavoidable.

## Depth Lower Bound and Correctness

The **depth** of a set of intervals is the maximum number of intervals covering any single point in time.

- **Lower bound:** every schedule needs at least depth classrooms — at that instant all those lectures are running and each needs its own room.
- **Greedy matches it:** suppose greedy opens its $d$-th classroom for lecture $j$. At that moment all $d-1$ existing rooms have finish time $> s_j$; by the start-time ordering their lectures started at or before $s_j$, so together with $j$ there are $d$ lectures covering time $s_j$. Hence depth $\geq d$.

Greedy uses exactly (no more than) the depth $\Rightarrow$ optimal.

## Running Time

- Naively scanning all rooms for compatibility: $O(n^2)$.
- Key observation: if $j$ is incompatible with the room whose last lecture finishes **earliest**, it is incompatible with every room. So one min-heap keyed by room finish times suffices: each lecture does one `Min` check ($O(1)$) and at most one `ExtractMin`/`Insert` ($O(\log n)$ each).
- Total: $\Theta(n \log n)$ including the sort.

## Sorting by Finish Time with Arbitrary Room Assignment Fails

Intervals $(1,3),(2,4),(4,5),(3,6)$, processed by increasing finish time:

1. $(1,3) \rightarrow$ room 1 (finishes 3).
2. $(2,4)$: no room free $\rightarrow$ room 2 (finishes 4).
3. $(4,5)$: both rooms are free; **arbitrarily** choose room 1 (finish 3) $\rightarrow$ room 1 now finishes 5.
4. $(3,6)$: neither room is free $(5, 4 > 3) \rightarrow$ room 3.

This uses 3 rooms although depth is 2 (optimal: room 1 gets $(1,3),(3,6)$; room 2 gets $(2,4),(4,5)$). **Fix (tie-breaking):** if several rooms are available, reuse the one with the **latest** finish time (best fit). Implement with a BST of rooms ordered by finish time: take the predecessor with finish $\leq s_j$ and update it to $f_j$; total $O(n \log n)$.

# 8 Huffman Coding

Huffman coding is another greedy algorithm: it builds an optimal **prefix-free binary code** for a file whose character frequencies are known.

## 8.1 Motivation: Variable-Length Encoding

There are over $10^6$ characters across all languages: a fixed-length code would need $\geq 21$ bits per character, while English needs only 7 (ASCII). Internet traffic is predominantly English, so **variable-length** codes — short codewords for frequent characters, long codewords for rare ones — save many bits in expectation.

## 8.2 Prefix-Free Codes

A code is a **prefix-free code** ("prefix code") if no codeword is a prefix of any other codeword.

**Prefix-Free ⇒ Unique Decodability**

**Theorem.** Every message encoded by a prefix-free code is uniquely decodable: reading left to right, the bits can match only one codeword (it cannot be a prefix of a longer one), so peel it off and continue.

**Examples.**

- $C_1 = \{a = 00, b = 01, c = 10, d = 11\}$ (fixed-length): $010011 = 01\,|\,00\,|\,11 \rightarrow$ "bad".
- $C_2 = \{a = 0, b = 110, c = 10, d = 111\}$ (prefix-free): $1100111 = 110\,|\,0\,|\,111 \rightarrow$ "bad".
- $C_3 = \{a = 1, b = 110, c = 10, d = 111\}$ (not prefix-free, since 1 is a prefix of 110 and 111): 1101111 can be parsed as $110\,|\,1\,|\,111$ = "bad" or $1\,|\,10\,|\,1\,|\,111$ = "acad" — ambiguous!

**Why Prefix Codes in Practice?**

UTF-8 marks continuation bytes with the prefix `10`. If bytes are corrupted or dropped, the decoder self-synchronizes: skip ahead to the next byte not beginning with `10` rather than losing the rest of the file.

## 8.3 The Encoding Problem

Given characters with known frequencies, find a prefix code minimizing the total encoded length (compression).

**Example file:** 100,000 characters over $\{a, b, c, d, e, f\}$ with frequencies in thousands:

| | $a$ | $b$ | $c$ | $d$ | $e$ | $f$ | Total (bits) |
|---|---|---|---|---|---|---|---|
| Frequency ($\times 1000$) | 45 | 13 | 12 | 16 | 9 | 5 | 100,000 chars |
| Fixed-length code | 000 | 001 | 010 | 011 | 100 | 101 | $3 \cdot 100{,}000 = 300{,}000$ |
| Variable-length code | 0 | 101 | 100 | 111 | 1101 | 1100 | 224,000 |

The variable code costs

$(45{\cdot}1{+}13{\cdot}3{+}12{\cdot}3{+}16{\cdot}3{+}9{\cdot}4{+}5{\cdot}4){\cdot}1000 = (45{+}39{+}36{+}48{+}36{+}20){\cdot}1000 = 224{,}000$ bits,

a 25% reduction.

## 8.4 Tree Correspondence and Weighted External Path Length

**Prefix Codes ↔ Binary Trees**

Represent a prefix code by a binary tree: label the left edge 0 and the right edge 1; the codeword of a character is the bit string on the root-to-leaf path.

- Every **leaf** stores one character; an internal node has two children (a "full" tree — no leaf codeword can be a prefix of another).
- Leaf depth $d(a_i)$ equals codeword length.
- Set leaf weight to the character frequency $f(a_i)$. The **weighted external path length**

$$B(T) = \sum_{i=1}^{n} f(a_i)\, d(a_i)$$

is exactly the total encoded length. Finding the minimum-cost prefix code = finding a minimum-weighted-external-path-length binary tree!

In the example, the fixed-length tree (depth 3 for all six leaves) costs $(45{+}13{+}12{+}16{+}9{+}5){\cdot}3 = 300$ (thousand bits); the variable tree costs $45{\cdot}1{+}(12{+}13{+}16){\cdot}3{+}(9{+}5){\cdot}4 = 224$.

## 8.5 Huffman's Greedy Algorithm

**Greedy idea (a "merge"):**

1. Pick the two characters $x, y$ with the **smallest** weights.
2. Join them as sibling leaves under a new internal node $z$ with weight $f(z) = f(x)+f(y)$.
3. Remove $x, y$ from the alphabet and insert $z$.
4. Repeat until one node (the root) remains.

**Algorithm 31** Huffman

```
function HUFFMAN(A)
    build a min-priority queue Q on A, keyed by weight
    for i = 1 to n − 1 do
        allocate a new node z
        x ← EXTRACTMIN(Q)
        y ← EXTRACTMIN(Q)
        z.left ← x;    z.right ← y
        z.weight ← x.weight + y.weight
        INSERT(Q, z)
    end for
    return EXTRACTMIN(Q)                    ▷ root of the Huffman tree
end function
```

**Running time:** $O(n \log n)$ with a binary min-heap ($n-1$ merge rounds, each with two extract-mins and one insert, each $O(\log n)$).

## 8.6 Worked Example

Initial alphabet (weights): $f$:5, $e$:9, $c$:12, $b$:13, $d$:16, $a$:45.

| Round | Merge | Priority queue afterwards |
|---|---|---|
| 0 | — | $\{5f,\ 9e,\ 12c,\ 13b,\ 16d,\ 45a\}$ |
| 1 | $5f + 9e = 14$ | $\{12c,\ 13b,\ 14,\ 16d,\ 45a\}$ |
| 2 | $12c + 13b = 25$ | $\{14,\ 16d,\ 25,\ 45a\}$ |
| 3 | $14 + 16d = 30$ | $\{25,\ 30,\ 45a\}$ |
| 4 | $25 + 30 = 55$ | $\{45a,\ 55\}$ |
| 5 | $45a + 55 = 100$ | $\{100\}$ (root) |

Reading root-to-leaf paths (left = 0, right = 1) gives:

| Character | $a$ | $c$ | $b$ | $f$ | $e$ | $d$ |
|---|---|---|---|---|---|---|
| Codeword | 0 | 100 | 101 | 1100 | 1101 | 111 |
| Length | 1 | 3 | 3 | 4 | 4 | 3 |

**Two Ways to Compute the Total Cost**

- By leaf depths: $B(T) = \sum_i f_i d_i = 45{\cdot}1 + (12 + 13 + 16){\cdot}3 + (9 + 5){\cdot}4 = 224$.
- By merge weights: $B(T) = 14 + 25 + 30 + 55 + 100 = 224$.

The sums agree because a leaf of weight $f$ contributes $f$ to the merge weight of every internal node on its path to the root — exactly one contribution per level of depth. This identity is the key to both the proof and the optimal-merge application.

## 8.7 Correctness Proof

**Lemma 1: An Optimal Tree Is Full**

Every internal node in an optimal prefix-code tree has exactly two children. If an internal node had only one child, contract that node (replace it by its child): no leaf gets deeper, and some leaves get shallower, strictly decreasing $B(T)$ — contradiction.

**Lemma 2: Swapping Leaves**

Let $T'$ be obtained from $T$ by swapping two leaves $x$ and $b$. If $f(x) \le f(b)$ and $d(x) \le d(b)$ (move the smaller-frequency leaf down and the larger one up), then

$$B(T') = B(T) + \big(f(x) - f(b)\big)\big(d(b) - d(x)\big) \le B(T),$$

since the first factor is $\le 0$ and the second is $\ge 0$. So placing frequent characters nearer the root never increases the cost.

**Lemma 3: The Two Smallest Weights Can Be Siblings at Maximum Depth**

Let $x, y$ be the two least frequent characters. Take any optimal tree $T$ (full by Lemma 1); it contains two sibling leaves $b, c$ at the deepest level. Assuming $f(x) \le f(b)$ and $f(y) \le f(c)$ (relabel if needed), swap $x \leftrightarrow b$ and $y \leftrightarrow c$. By Lemma 2 the cost does not increase; since $T$ was optimal, the resulting tree is also optimal and now has $x, y$ as sibling leaves at the deepest level.

**Lemma 4: Contracting Two Sibling Leaves**

If $x, y$ are sibling leaves in $T$, contract them into one leaf $z$ with $f(z) = f(x) + f(y)$ to obtain $T'$, then

$$\begin{aligned} B(T) &= B(T') - f(z)d(z) + f(x)(d(z) + 1) + f(y)(d(z) + 1) \\ &= B(T') + f(x) + f(y). \end{aligned}$$

**Theorem: Huffman Produces an Optimal Tree (Induction on $n$)**

**Base ($n = 2$):** the only full tree has two leaves at depth 1; it is optimal.
**Inductive step:** assume Huffman is optimal for alphabets of size $n - 1$. For size $n$, let $H$ be the Huffman tree and $x, y$ its first merge (the two smallest weights, siblings in $H$). Contract them to $z$ with $f(z) = f(x) + f(y)$: the contracted tree $H'$ is exactly the tree Huffman builds on the $n - 1$-character alphabet $A' = A - \{x, y\} \cup \{z\}$, so by the induction hypothesis $H'$ is optimal for $A'$, and by Lemma 4

$$B(H) = B(H') + f(x) + f(y).$$

By Lemma 3 there is an optimal tree $T$ for $A$ with $x, y$ as siblings; contract them to get $T'$ for $A'$. Then

$$B(H) = B(H') + f(x) + f(y) \leq B(T') + f(x) + f(y) = B(T).$$

Since $T$ is optimal, $H$ is optimal too. □

**Huffman Caveats**

- Huffman is optimal **among prefix codes**, not among all possible codes (arithmetic coding can exploit fractional code lengths).
- Each merge must join the two current smallest weights — joining the smallest with an arbitrary node is not optimal.
- Ties in weight may be broken arbitrarily: different trees can result, all with the same minimum cost.
- Edge case $n = 1$: the single character gets the empty codeword (conventionally one bit may be used).

## 8.8 Application: Optimal Two-Way Merge

Given $n$ sorted lists $L_1, \ldots, L_n$, merge them into one sorted list, merging only two lists at a time; merging lists of sizes $p$ and $q$ costs $p + q$ comparisons and produces a list of size $p + q$. Find the merge pattern minimizing total comparisons.

A merge pattern is a binary tree: leaves are the initial lists (weight = list size), each internal node is a merge, and the total cost equals the **weighted external path length** — so Huffman (always merge the two smallest lists) is optimal.

**Example:** three lists of sizes $30, 20, 10$:

| Pattern | Computation | Total comparisons |
|---|---|---|
| $L_1, L_2$ first, then $L_3$ | $30 + 20 = 50$, then $50 + 10 = 60$ | $50 + 60 = 110$ |
| $L_1, L_3$ first, then $L_2$ | $30 + 10 = 40$, then $40 + 20 = 60$ | $40 + 60 = 100$ |
| $L_2, L_3$ **first, then** $L_1$ **(Huffman)** | $20 + 10 = 30$, then $30 + 30 = 60$ | $\mathbf{30 + 60 = 90}$ |

**Algorithm 32** Optimal Binary Merge Tree

```
build a min-heap from the n initial list sizes
while heap size ≥ 2 do
    a ← ExtractMin;   b ← ExtractMin
    create an internal node of size a + b with children a, b
    Insert(a + b)
end while
```

Time $O(n \log n)$.

**Worked example:** initial sizes $2, 3, 5, 7, 9$:

| Round | Merge | Heap afterwards |
|---|---|---|
| 1 | $2+3=5$ | $\{5,5,7,9\}$ |
| 2 | $5+5=10$ | $\{7,9,10\}$ |
| 3 | $7+9=16$ | $\{10,16\}$ |
| 4 | $10+16=26$ | $\{26\}$ |

Total cost (sum of merge sizes): $5+10+16+26 = \mathbf{57}$, which equals the weighted external path length of the merge tree:

$$2\cdot 3+3\cdot 3+5\cdot 2+7\cdot 2+9\cdot 2 = 6+9+10+14+18 = 57.$$

**Greedy Algorithms at a Glance**

| Problem | Greedy choice | Complexity | Correctness idea |
|---|---|---|---|
| Interval scheduling | earliest finish time | $\Theta(n \log n)$ | exchange argument |
| Interval covering | furthest reach covering frontier | $O(n \log n)$ | exchange argument |
| Fractional knapsack | largest value/weight | $\Theta(n \log n)$ | exchange argument |
| Interval partitioning | earliest start, reuse a room | $\Theta(n \log n)$ | match depth lower bound |
| Huffman coding | merge two smallest weights | $O(n \log n)$ | exchange + induction |
| Optimal merge | merge two smallest lists | $O(n \log n)$ | same as Huffman |